MIZAN: Measure-First Adaptive Inference for Energy-Constrained Satellite Payloads

IASTAM 6.0 Technical Challenge — Artificial Intelligence and Onboard Computing: AI Inference Under Energy Constraints

Ahmed Hedhili, Salem Essid, Ahmed Mnasri
Team INTERSTELLAR
École Supérieure des Communications de Tunis (SUP'COM), University of Carthage, Tunisia

***Abstract*—A satellite that runs AI onboard must decide, frame by frame, how much of a small and shifting energy, memory and time budget each inference deserves. We present MIZAN (Arabic for *balance*), a measure-first adaptive inference system for energy-constrained payloads, and use it to test the three levers such a system has: which model to run, how often to run it, and how much detail to process. MIZAN pairs a calibrated early-exit network at two resolutions with a controller whose safety filter can veto a configuration but never choose one, and which prices energy in closed form from a profiled cost menu, corrected online. Three findings follow. First, adapting the model saves 52–57% of per-inference energy on three data-centre accelerators while saturated, and nothing measurable at ten frames per second, because the platform's fixed draw dominates a paced frame: on a duty-cycled payload the energy lever is how often to run, not how deep. Second, a cost menu profiled on a development laptop is $6.5\times$ optimistic on our Raspberry Pi 4B payload, the computing board of the BUPT-1 CubeSat, measured floor to floor. Third, correcting that menu online against observed cost cuts mean latency by 31% and missed deadlines by 49% ($p = 0.0065$) on the board, for 2.7 points of menu-ascribed accuracy; on today's narrow menu the cheapest fixed option still misses fewer, which sets our next step. Adaptive inference onboard must measure the platform it runs on, and keep measuring it.**

***Index Terms*—onboard AI inference, energy-constrained computing, early-exit neural networks, resource-aware adaptive control, orbital edge computing**

I. Introduction

Compute is moving to orbit, and with it the question of how a satellite should spend the energy it has. A payload that images continuously cannot downlink everything, so it must decide onboard what each frame is worth, under constraints that move: the battery charges and discharges with the orbit, the processor heats, and a competing task takes the deadline away.

This paper addresses *AI Inference Under Energy Constraints*, the problem statement of the Artificial Intelligence and Onboard Computing track of the IASTAM 6.0 Technical Challenge. A strategy can adapt along three levers — *which model* to run, *how often* to run it, and *how much detail* to process — and is judged on accuracy, inference time, energy consumption, memory used, and robustness to varying resources.

Adaptive inference is usually evaluated on a development machine rather than the platform that would fly it, and its energy is either not measured [1] or not separated into the board's fixed draw and the share a policy controls [2]. We build **MIZAN** (Arabic for *balance*) to measure both: an inference stage offers operating points whose costs are measured on the target, and a controller picks among them under the resources of the moment, without ever being able to override a safety limit. Our target is a Raspberry Pi 4B ($8\,$GB), the general-purpose computing payload of the BUPT-1 CubeSat [3], [4], where in-orbit measurement finds temperature and energy, not compute, to be the binding constraints [5]. Three data-centre accelerators, the hardware class orbital data-centre proposals assume [6], [7], give a contrasting regime. We make three contributions.

1) **A measurement-first methodology**: an early-exit network at two resolutions whose gated execution reproduces fixed-exit outputs exactly, a data-disjoint certification with a stated selective-error guarantee, and a provenance label on every quantity. It finds a host-profiled cost menu about $6.5\times$ optimistic on the target board, floor to floor (Sections III-D and IV-F).
2) **A veto-only controller**: a safety filter that can refuse a configuration but never choose one, an energy price computed onboard in closed form with no tuning constant, and cost feedback that corrects the menu online with a latency bound of stated miss rate (Sections III-B, IV-D and IV-H).
3) **A regime map for the three levers** (Fig. 1): on a duty-cycled payload, adapting the model buys deadline safety, not energy, and only with a cost model measured and corrected on the target. On the board, cost feedback cuts mean latency by 31% and missed deadlines by 49% for 2.7 accuracy points, though the cheapest fixed option misses fewer still (Sections IV-C and IV-G).

II. Related Work

Onboard inference. Φ-Sat-1 flew a convolutional network on a Myriad 2 vision processor to discard cloudy tiles [8]; surveys map onboard image processing [9] and orbital edge computing [10], [11]; Kodan fits every tile to the frame deadline and measures time but not energy [12]; Equinox runs a task only when its value exceeds a barrier-shaped cost of battery, temperature and queue state, across a simulated

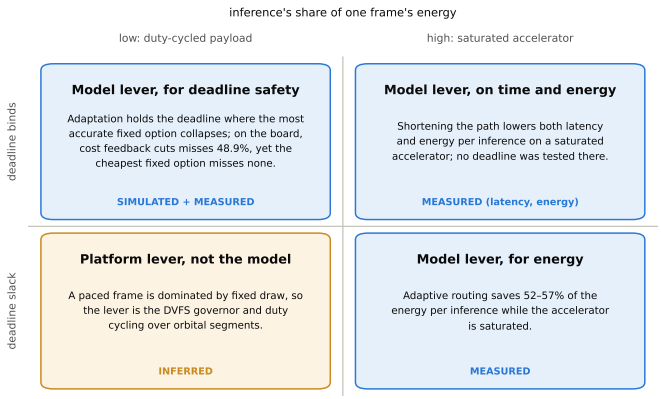


Fig. 1. The regime map: which lever pays depends on inference's share of one frame's energy and on whether the deadline binds. Badges give each cell's evidence: MEASURED on the stated platform, SIMULATED from declared assumptions, or INFERRED from a model and measurements elsewhere, not observed on the target board.

constellation [13]. We ask how a workload that is onboard should execute as its budgets move.

Dynamic and resource-aware inference. Early-exit networks [14], [15], [16], [17] adapt to the *input*, not to the platform, and report FLOPs or latency; a recent comparison on a Jetson Orin and a Raspberry Pi measures no energy [1], and E3C meters early exits on a Raspberry Pi 4B without separating the board's fixed draw from the share a policy controls [2]. NestDNN [18] and E4 [19] adapt to the resources available, but none bounds what its controller may do when the cost model is wrong. ReGate attributes 30–72% of neural-processing-unit energy to static power [20], energy no policy that only chooses how much to compute can see.

Risk and safety filters. Temperature scaling [21] and Learn then Test [22], [23] give calibrated and certified confidence. The Simplex architecture [24], shields [25] and predictive safety filters [26] let a layer veto a controller but never choose for it. We add where the filter's numbers come from — costs measured on the target and corrected online with an adaptive conformal bound [27] — and an energy price, the Lagrange multiplier of a budgeted sequential decision [28], [29], in closed form because the menu states the rewards, where a barrier-shaped cost [13] needs hand-set coefficients.

The gap (Table VII, Appendix A). No prior work measures what adaptation is worth in energy on a duty-cycled flight-class board, and none closes measured costs, resource-aware control and platform energy accounting into one loop under a filter that can only veto.

III. Methodology

A. Problem Formulation

Time is divided into frames $t = 0, \dots, T-1$ of period Δ (5 s in most of our scenarios, 0.1 s in the two whose energy binds hardest). At every frame the system either skips or runs one configuration a from a menu $\mathcal{A}$ of K options profiled offline, each with accuracy $Q(a)$, mean and 95th-percentile latency $\bar{\ell}(a)$ and $\ell_{95}(a)$, resident memory $m(a)$ and energy per inference $E(a)$, estimated as $P_{\text{act}}\,\bar{\ell}(a)$ from a declared active power where it has not been measured on the target. The frozen menu has $K = 8$ options of one early-exit network, with accuracy between 69.3% and 90.7% and ℓ_{95} between 2.8 and 9.6 ms on the development host; Phase 3 runs its certified successor (Table I).

The controller observes $s_t = (b_t, \theta_t, m_t, d_t)$ — stored energy, processor temperature, free memory and time budget — and chooses $a_t \in \mathcal{A} \cup \{\varnothing\}$, where SKIP ($\varnothing$, $Q(\varnothing) = 0$) is a first-class action. With τ_t the controller's decision time and switching costs from measured transitions, the frame is charged

$$E_t = E(a_t) + P_{\text{act}}\,\tau_t + E_{\text{sw}}\,\mathbf{1}[a_t \neq a_{t'}], \tag{1}$$

$$\ell_t = \ell_{95}(a_t) + \tau_t + \ell_{\text{sw}}\,\mathbf{1}[a_t \neq a_{t'}]. \tag{2}$$

Panels deliver $P_{\text{gen}}(t)$ outside eclipse; housekeeping P_{hk} and the value detector E_{det} draw on every frame, skipped or not:

$$\begin{aligned} b_{t+1} &= \max\{0,\ \hat{b}_t - E_t - P_{\text{hk}}\,\Delta - E_{\text{det}}\}, \\ \hat{b}_t &= \min\{B,\ b_t + \eta\,P_{\text{gen}}(t)\,\Delta\}, \end{aligned} \tag{3}$$

$$\begin{aligned} \theta_{t+1} &= \theta_\infty + (\theta_t - \theta_\infty)\,e^{-\Delta/\tau_{\text{th}}}, \\ \theta_\infty &= \theta_{\text{amb}}(t) + R\,\big((E_t + E_{\text{det}})/\Delta + P_{\text{hk}}\big). \end{aligned} \tag{4}$$

The objective is value-weighted accuracy under hard constraints:

$$\begin{aligned} \max_{a_0,\dots,a_{T-1}} \quad & \sum_{t=0}^{T-1} v_t\,Q(a_t) \\ \text{s.t.} \quad & b_{t+1} \geq b_{\min}, \quad \theta_{t+1} \leq \theta_{\max}, \\ & \ell_t \leq d_t, \quad m(a_t) \leq m_t, \quad \forall t. \end{aligned} \tag{5}$$

The value v_t is 1 when the frame images a target and 0.1 when it does not, seen only through a declared detector right on 90% of frames; results are scored with the true value, and skipped or vetoed frames count as errors. The menu carries the model and processing-quality levers, SKIP the frequency lever; SKIP does not power the payload down, since the board's fixed draw sits in P_{hk} (Appendix B).

B. Controller Architecture

Fig. 2 shows the loop: the controller proposes, a deterministic safety filter disposes, and the executor runs the approved option. Every layer is $O(K)$ per frame and runs unchanged on the target board.

Safety filter. A proposal is approved only if it lies in

$$\begin{aligned} \mathcal{A}_{\text{safe}}(s_t) = \{a \in \mathcal{A} :\ & \theta_t < \theta_{\max} - \delta_\theta, \quad b_t > b_{\min}, \\ & \ell_{95}(a) \leq d_t - \delta_\ell, \quad m(a) \leq m_t, \\ & E(a) + \delta_E \leq b_t - b_{\min}\}; \end{aligned} \tag{6}$$

otherwise the frame becomes SKIP. The filter can veto but never choose, so no controller, learned or hand-designed, can bypass it. The margin $\delta_\ell = 2$ ms covers the largest measured switching overhead (1.25 ms over 56 transitions) plus decision time.

Pareto policy. The innermost layer proposes

$$a_t = \operatorname*{arg\,max}_{a \in \mathcal{A}_{\text{safe}}(s_t) \cup \{\varnothing\}} \big[\, \hat{v}_t\, Q(a) - \lambda\, E(a) \,\big], \tag{7}$$

with $\hat{v}_t$ the detector's estimate; SKIP scores zero, so an option runs only when it is worth its energy at price λ.

Pricing energy without hindsight. λ is the Lagrange multiplier on the energy budget of the offline optimum (11). Tuning it needs foresight of the episode, but the menu states it. With H the episode length and d the declared orbit-average platform draw per frame (housekeeping plus detection, less average generation), each remaining frame may spend

$$\begin{aligned} T_t &= \frac{\max\{0,\ (b_t - b_{\min}) - (H-t)\, d\}}{H-t}, \\ a_t^\star &= \operatorname*{arg\,max}_{\{a\,:\,E(a) \le T_t\}} Q(a), \end{aligned} \tag{8}$$

where $a_t^\star$ is the best rung that allowance covers, or the cheapest when it covers none; when $(H-t)\, d$ exceeds the usable charge, the price drops to its minimum. The price at which no richer rung is worth its extra energy is

$$\lambda_t = \max_{\substack{a\,:\,E(a) > E(a_t^\star),\\ Q(a) > Q(a_t^\star)}} \frac{Q(a) - Q(a_t^\star)}{E(a) - E(a_t^\star)}, \tag{9}$$

kept at its previous value when no such rung exists. Both are recomputed every frame; H and d are declared mission parameters, not tuning constants.

Hysteresis holds the current option for $D = 3$ frames, released at once if it becomes infeasible. **Cost feedback** hands every inner layer the menu at its estimated cost: from the last $W = 100$ latencies $L_{a,1}, \ldots, L_{a,n}$ of each executed option it replaces $\ell_{95}(a)$ in (6) by the conformal bound

$$\hat{q}_a = L_{a,(k)}, \qquad k = \lceil (n+1)(1-\delta) \rceil, \tag{10}$$

which the next execution exceeds with probability at most $\delta = 5\%$ when latencies are exchangeable. A board whose clock changes is not, so adaptive conformal inference [27] moves the level after every frame, $\delta_{t+1} = \delta_t + \gamma\,(\delta - \mathbf{1}[L_t > \hat{q}_t])$ with $\gamma = 0.01$. An option executed fewer than 19 times keeps its menu cost.

Survival mode narrows the menu by state: SAVING to the cheaper half below 10 minutes of reserve at the unavoidable platform draw, CRITICAL to the cheapest option below 4 minutes, EMERGENCY to none; THERMAL_RECOVERY skips frames from 2 °C below the limit until the processor is 5 °C below it. Each layer can only narrow the choices of the layer inside it, and the filter keeps the final word.

C. Closed-Loop Evaluation Protocol

Every controller runs on a fresh copy of the same seeded environment, so differences come from decisions only, against four static references: always the most accurate option, always the cheapest, the most accurate option that fits every frame, and the best static option chosen with hindsight. The denominator is an *offline optimum*: relaxing the per-frame battery constraint to the episode's energy budget B gives a Lagrangian that separates across frames,

$$L(\lambda) = \sum_t \max_{a \in \mathcal{A}_t} \big[v_t Q(a) - \lambda E(a) \big] + \lambda B, \tag{11}$$

whose minimum over $\lambda \ge 0$ upper-bounds the constrained optimum by weak duality; bisection finds λ, and replaying its policy brackets the optimum. It is tight on five of seven scenarios (Appendix C); a second ceiling, matched to the safety filter, is used only where stated (Appendix B). Metrics follow the five indicators: per-frame accuracy, ℓ_{95} frame latency including controller and switching time, inference, controller and switching energy (measured, or labelled estimated), resident memory, and robustness as quality relative to the optimum while resources vary. Violations are counts, split into unavoidable ones and those caused by decisions.

D. Adaptive Inference Model and Its Menu

The task is EuroSAT RGB land-use classification [30], a standard proxy for an Earth-observation payload that labels what it images before deciding what to keep. A single MobileNetV3-Small backbone [31], initialised from ImageNet weights, carries three classifiers at increasing depth and runs at two input resolutions (48 and 64 pixels), so adaptation has three knobs that cost nothing to store: where to stop, how much input detail to use, and whether to run at all. All weights stay resident, so memory barely differs across options. Each head is temperature-scaled [21] on a split training never saw; image x leaves at the first exit k with $\max_c p_k(c \mid x) \ge \lambda_k$, and a gated run costs exactly the path it took, verified on thousands of images with zero mismatches.

Thresholds with a stated guarantee. With $\alpha = \delta = 0.10$ declared before any evaluation, candidate policies are tested in a fixed order on a certification split no earlier stage touched, by exact binomial tests of the *selective* error $\Pr(\text{wrong} \mid \text{exited early})$, stopping at the first non-rejection [22], [23]. The survivor carries the statement that, with probability at least $1-\delta$ under exchangeability with the certification region, images that leave early are wrong at most α of the time. Every option's accuracy comes from images that neither trained the model nor selected its thresholds, and its costs from running that exact checkpoint.

Two menus. Most results use the frozen eight-option menu of one training run (Table VIII, Appendix B). A pre-registered retraining over five seeds found that gating costs about 1.7 points at 64 pixels and that a geographic split costs about five; its certified successor reaches 90.15% against 91.02% at full depth (not significant) at under half the mean latency, and Phase 3 runs five of its rungs (Table I).

IV. Experiments and Results

A. Testbed, Simulation Setup and Provenance

The **target board** is a Raspberry Pi 4 Model B (8 GB), 64-bit Linux, running inference on its ARM CPU; it ran four sessions, three with the desktop active on synthetic images and one headless on real EuroSAT images. A laptop CPU,

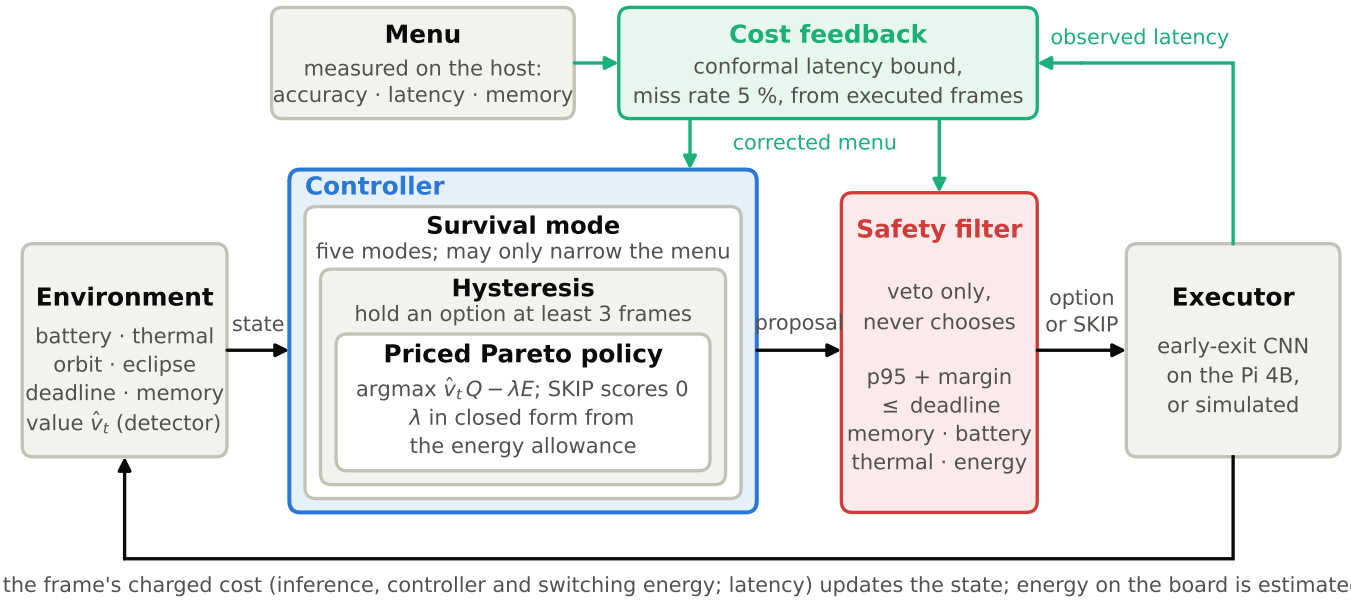


Fig. 2. MIZAN system architecture. Survival mode wraps hysteresis, which wraps the Pareto policy priced in closed form (9); together they propose an option, and the safety filter can only veto it. Cost feedback hands the controller and the filter the menu at its corrected cost (10), learned from executed frames only.

TABLE I
THE MENU PHASE 3 AND THE HELD-OUT EVALUATION RUN: FIVE RUNGS OF THE CERTIFIED SUCCESSOR. ACCURACY ON 1,625 IMAGES OF A GEOGRAPHICALLY HELD-OUT REGION; COSTS ON THE DEVELOPMENT HOST, NOT YET RE-PROFILED ON THE BOARD.

Res.	Rung	Acc. (%)	Mean (ms)	ℓ_{95} (ms)	RSS (MB)
64	exit 1	83.88	1.55	2.08	307
64	adaptive, depth cap 2	87.63	1.86	3.38	298
48	adaptive, depth cap 2	88.31	2.09	3.72	308
64	adaptive, certified	90.15	1.91	4.25	308
64	full depth	91.02	4.58	5.51	306

the **development host**, profiled the menu. The **accelerators** are an NVIDIA A40 and cloud L4 and T4 instances with their own power telemetry, at batch size one. The **simulator** models a battery with a floor, first-order thermal relaxation, eclipse-gated solar generation and housekeeping charged on every frame, in seven named scenarios each built so a stated resource binds; Phase 3 adds *energy_trade* and two held-out scenarios. Every quantity is labelled *measured*, *simulated*, *estimated* or *declared* (Table X, Appendix E); board energy is not yet measured, since no session had a meter. Table II summarises the findings.

B. Result: What Adaptive Inference Costs on Real Accelerators

Each option runs alone on an accelerator, in randomised order, across repeated windows, either saturated or paced at a fixed frame rate. Per-inference energy is the board's power-counter difference over a window divided by completed inferences, idle draw included (Table III); subtracting an idle baseline measured inside every window isolates E_{dyn}, the energy inference itself adds.

Table III shows a large, reproducible saving on a saturated accelerator and none at 10 frames per second on the same hardware. Per frame,

$$E_{\text{frame}}(f) = \frac{P_{\text{idle}}}{f} + E_{\text{dyn}}. \tag{12}$$

E_{dyn} is at most 20.2 mJ at full depth (L4, above idle), against P_{idle}/f of 2.8–7.2 J at $f = 10\,\text{s}^{-1}$, so the model's share of a paced frame is about one percent or less; measured in the paced runs at full depth it is 1.2% on the L4 and indistinguishable from zero on the T4. The saturated saving is throughput, not lower power: the idle draw is 94.1%, 87.2% and 97.5% of a saturated full-depth inference on the A40, L4 and T4, and 86–97% of the saving is idle power charged over a shorter window. On a duty-cycled payload, adapting the model is not an energy lever; adapting the *frequency*, or powering down between bursts, is.

C. Result: When Adapting Pays on Energy

What adaptation can save is bounded by inference's share of the energy the platform spends anyway. We sweep that share by varying housekeeping power on one scenario and score each controller as a share of the offline optimum (Fig. 3a). Where the energy budget binds, a price tuned with hindsight reaches 96.7–99.0%, against 96.7–97.6% for the best fixed configuration with the same hindsight: adaptation is at best a couple of points ahead. The price computed onboard matches or beats the tuned one at the two looser binding points (99.3% and 98.4%) and falls short at the tightest (91.7% against 96.7%), where every controller, fixed ones included, spends through the battery floor (rings). The accelerators say the same from the hardware side (Fig. 3b). The case for adapting

TABLE II

SUMMARY OF FINDINGS. EVIDENCE STATUS USES THE WORDS OF THE PROVENANCE TABLE (TABLE X, SECTION IV-A). EACH ROW IS STATED IN FULL IN THE SECTION NAMED.

#	Finding	Key number	Evidence status	Section
F1	The model lever saves energy only when the accelerator is saturated	52–57% saturated; indistinguishable from 0 at 10 fps	measured (GPU boards)	IV-B
F2	A host-profiled menu is optimistic on the board	about 6.5× at the floor	measured (Pi 4B)	IV-F
F3	Cost feedback recovers deadlines on the board	−48.9% misses ($p = 0.0065$), −31.1% mean latency ($p = 0.0011$)	measured (Pi 4B)	IV-G
F4	The cheapest fixed option still misses fewer deadlines	0 vs 23 misses; 0 again when interleaved (session 4)	measured (Pi 4B)	IV-G
F5	The onboard energy price needs no hindsight where energy binds	98.4% of optimum; hindsight price and cheapest option 97.6%	simulated	IV-D
F6	Inference is a few percent of a paced frame's energy on the board	0.2–3.4%	estimated	IV-F
F7	The onboard price removes the breaches a hand-set price causes	454 (policy) and 154 (full stack) to 0	simulated	IV-H
F8	The board makes the host's decision on every frame	2,560 of 2,560 frames, real images	measured (Pi 4B)	IV-F

TABLE III

MEASURED TOTAL ACCELERATOR ENERGY PER INFERENCE AT 64 PIXELS, IDLE DRAW INCLUDED, ADAPTIVE ROUTING AGAINST FULL DEPTH, WITH 95% CONFIDENCE INTERVALS OVER THREE TRIALS. SATURATED: THE ACCELERATOR IS THE BOTTLENECK; PACED: 10 FRAMES PER SECOND, BOARD POWERED BETWEEN THEM.

	A40	L4	T4
Idle board power (W)	71.9	28.6	28.0
Full depth, sat. (mJ)	324.4	157.2	157.8
Adaptive, sat. (mJ)	151.5	67.7	75.8
Saving, sat. (%)	53.3	56.9	52.0
95% interval	[50.4, 56.2]	[56.2, 57.6]	[42.3, 61.6]
Saving at 10 FPS (%)	–	0.65	−0.05
95% interval	–	[−2.1, 3.4]	[−0.2, 0.1]

TABLE IV

WHERE THE ENERGY PRICE COMES FROM, ON THE ONE SCENARIO WHERE ENERGY BINDS (OFFLINE OPTIMUM 1021.09; 3,000 FRAMES). BREACHES ARE DECISION-CAUSED. THE INTERIM RULE MEASURED THE DRAW OVER THE WHOLE EPISODE; THE CURRENT ONE DECLARES IT AND DIVIDES THE CHARGE LEFT BY THE FRAMES LEFT.

Price source	Quality	% opt.	Exec.	Breaches
untuned default	890.09	87.2	2547	454
tuned with hindsight	996.16	97.6	3000	0
computed onboard, interim rule	996.17	97.6	3000	0
computed onboard, current rule	1004.81	98.4	3000	0

onboard therefore rests on deadlines under changing resources, not on energy.

D. Result: The Energy Price Need Not Be Tuned

Where energy binds, an untuned default price reaches only 86.7–93.5% of the offline optimum, and the price that does better needs hindsight. Table IV compares the same decision policy with three price sources on the one scenario of our seven where energy binds. The onboard price of (8)–(9) matches the hindsight price (97.6%, every frame executed, no breach) where the default loses hundreds of frames to the filter, and under the current rule it reaches 98.4%, above hindsight, because it can move during the episode. Inside the *full* stack the same price reaches 93.2%, 4.4 points lower: survival mode and hysteresis buy constraint safety the price already bought. The result shows the rule finds the right rung without hindsight, not that adapting pays: running the cheapest option unconditionally scores within a fraction of a point. On *energy_trade*, built so the optimum moves between three rungs, always running the certified rung reaches 99.4% and the full system 98.3% (99.3% with a perfect value detector): on a menu whose certified rung is within 0.9 points of full depth at 42% of its energy, adapting is worth under a point.

E. Result: Robustness to a Varying Time Budget and to a Wrong Menu

A varying time budget. With energy and memory non-binding, seeded bursts tighten the deadline over part of a 600-frame run (Fig. 7, Appendix F). The adaptive controller matches the per-frame best score in every setting; the most accurate fixed option drops from 90.7% to 63.5% accuracy at 30% of frames under pressure, and to 0% once pressure is constant, because the filter refuses it; the best fixed option chosen with hindsight survives only by always running the fastest option.

A wrong menu. When execution is slower than the menu says, the filter approves work that cannot finish in time (Table XI, Appendix F). At a 1.5× slowdown a menu-blind controller misses all 200 contended deadlines, so its on-time accuracy is 60.4%; with cost feedback it misses 2 and keeps 86.4% on time. At the 8–9× of the contended board no option fits and the only safe action is to skip: cost feedback buys *deadline safety, not quality*.

F. Result: The Target Board — Transfer Error and Platform Behaviour

The menu is optimistic by about 6.5×, floor against floor. The cheapest option, exit 1 at 64 pixels, runs on the board at a first quartile of 9.50 ms in one session and 9.41 ms twenty hours later — a floor reproducible to 0.9% — against 1.45 ms on the re-measured host: 6.56× and 6.50×. Between the same sessions the median moved 56% on ambient load

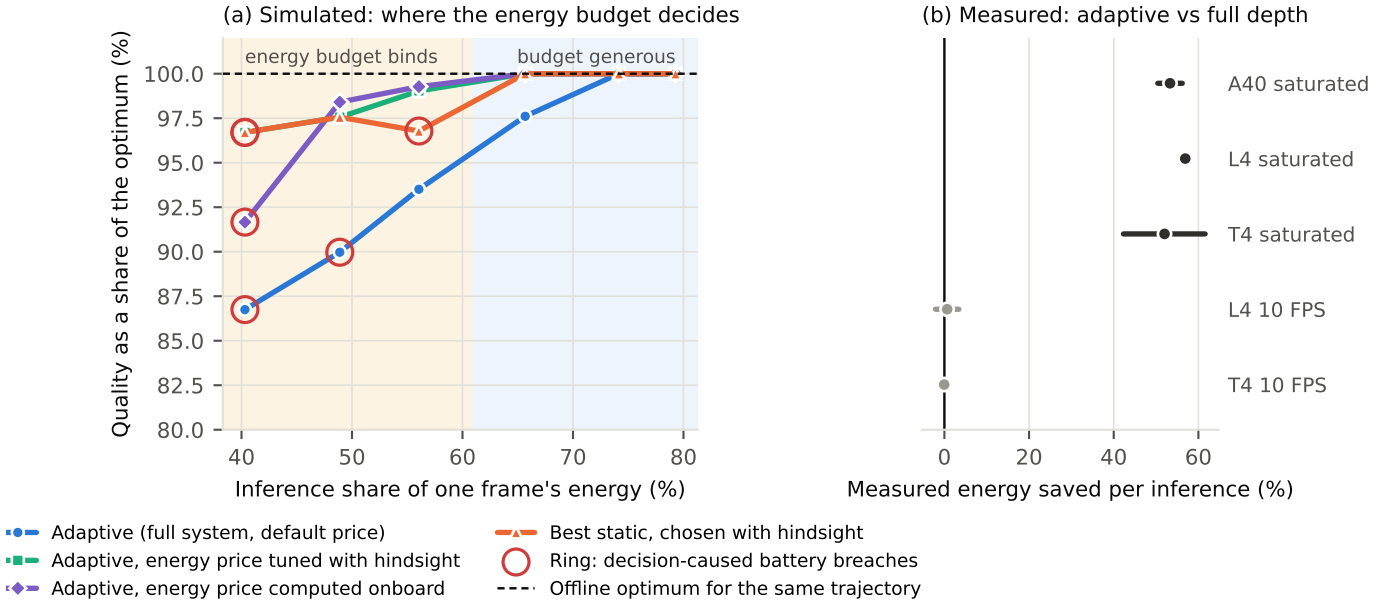


Fig. 3. When adapting pays on energy. (a) Simulated on *compute_bound* with the frozen menu: quality as a share of the offline optimum against inference's share of a frame's energy (axis from 80%); a ring marks decision-caused battery breaches. Below 40% housekeeping alone breaches the floor; the board's estimated share (0.2–3.4%, Section IV-F) lies there. (b) Measured on three accelerators: total energy saved per inference by adaptive routing, 95% intervals; the A40 was not run paced.

alone (Table XIII, Appendix H), so a menu entry, which claims what the work costs, is tested against uncontended frames on both machines. A ratio this far beyond the $1.5\times$ that already defeats a menu-blind controller makes cost feedback a precondition for the margins of (6) to mean anything.

Session 4: headless, real images. The floor ratio is $6.60\times$. The slow tail came from the CPU governor: for twenty minutes the clock sat at 600–900 MHz and every arm ran about $1.9\times$ slower. Deadline misses followed the clock, not the controller: 15 and 12 for the two adaptive arms in the slow block, none at full clock, and none for the cheapest option. On all 2,560 frames, across 7 options, the board predicted the host's class and took the host's exit, so the host's accuracy carries over; the paced images covered 2 of the 10 classes.

Decision cost, heat and the energy share. Run means are 0.02–0.03 ms per decision for a fixed option, 0.08–0.13 ms for the adaptive policy and 0.35–0.55 ms with cost feedback, whose 1.19 ms maximum plus a 1.25 ms switch crossed the 2 ms margin on 30 of 360 frames at the low clock (Appendix D). No sample throttled against an 80 °C limit (hottest 68.1 °C). From a published Pi 4 base draw of 1518 mW [32], our declared 2.0 W active increment, the 5 s frame and the largest mean latency any session produced (26.95 ms), one inference is 0.7% of a frame's energy, and 0.2–3.4% across any plausible mismatch: *estimated*, not measured, but it agrees with the accelerators.

G. Result: Cost Feedback on the Target Board

A second board session drives the controller with the board's own transfer error, cost feedback disabled and enabled in interleaved runs (Table VI). Feedback cuts mean latency by 31.1% and deadline misses by 48.9%, for 2.7 accuracy points; every trial with feedback beats every trial without it on mean latency ($p = 0.0011$, exact permutation test, six trials per arm), and misses fall from 7.5 to 3.8 per trial ($p = 0.0065$). Without it the controller stayed on one option budgeted at 6 ms and executed at 26–27 ms; with it, it spread across seven. On real images in session 4 it helped only while the clock was low: 13.8% lower mean latency and misses from 15 to 12, for 2.22 measured accuracy points ($p = 0.008$, McNemar).

From an average to a bound. That rule learned an exponential average of observed over predicted latency, which settles near the mean and understates the tail: replaying session 4 in time order, its bound was exceeded on 33.2% and 36.5% of frames. The adaptive conformal bound (10), which the controller now runs, is exceeded on 5.0% and 5.1% against its 5% budget, across the clock change.

Against the cheapest fixed option. Always running the cheapest option missed no deadline in either session. What the corrected controller buys over it, about 1.5 accuracy points, costs 23 missed deadlines. The cause is the menu: with a narrow accuracy spread, adaptation has little to win and a latency tail to risk. These sessions run the scenario where nothing binds; in simulation the comparison inverts once a resource binds (Section IV-H).

H. Ablation, and the System the Held-Out Run Uses

What each layer buys. Removing one layer of Survival(Hysteresis(Pareto)) at a time, filter always on (Table XII, Appendix F), changes nothing where no resource binds. Where one binds, survival mode buys constraint satisfaction, not quality: on *thermal_ramp*, where most breaches are

TABLE V

THE SYSTEM M4 RUNS, ON THE EIGHT DEVELOPMENT SCENARIOS: SHARE OF THE OFFLINE OPTIMUM (%). *BEFORE A3*: THE INTERIM PRICE RULE. *MATCHED*: DIVIDED BY THE CEILING MATCHED TO THE SAFETY FILTER. *NO LEVELS* (NO SAVING OR CRITICAL) AND *ACCOUNTED* (ALSO COUNTING THE CONTROLLER'S DECISION ENERGY IN THE PRICE'S DRAW) ARE EXPLORATORY, NOT PART OF M4. NO COLUMN CAUSES A DECISION-CAUSED BREACH AT THE DECLARED 0.02 MS DECISION TIME. SIMULATED; ENERGY ESTIMATED AT THE DECLARED 2 W.

Scenario	M4	Before A3	Matched	Best fixed	Cheapest	No levels	Accounted
stable	100.0	100.0	100.0	100.0	92.2	100.0	100.0
orbital	100.0	100.0	100.0	100.0	92.2	100.0	100.0
draining	64.6	47.7	94.1	68.6	63.3	68.6	68.6
thermal ramp	64.9	64.9	76.5	84.7	78.1	64.9	64.9
sudden drop	99.3	99.3	100.0	96.5	92.4	99.3	99.3
deadline pressure	99.6	99.6	100.0	96.4	92.3	99.6	99.6
compute bound	93.1	70.4	93.1	99.9	93.0	100.0	100.0
energy trade	98.3	98.0	98.3	99.4	92.5	98.3	98.3

TABLE VI

COST FEEDBACK ON THE TARGET BOARD (SECOND SESSION, INTERLEAVED, 720 FRAMES PER ARM), BESIDE THE CHEAPEST FIXED OPTION (FIRST SESSION, 360 FRAMES, SAME SCENARIO, RATE AND MENU, NOT INTERLEAVED). ACCURACY IS THE MENU'S VALUE FOR THE OPTIONS CHOSEN (SYNTHETIC IMAGES).

	Fb. off	Fb. on	Change	Cheapest
Mean latency (ms)	26.72	18.42	−31.1%	12.37
ℓ_{95}, pooled (ms)	52.53	38.20	−27.3%	22.25
Deadline misses	45	23	−48.9%	0
Options used	1	7	—	1
Expected accuracy	0.9065	0.8794	−2.71 pp	0.8645

unavoidable, the thermally blind variant scores 84.7% against 64.9%; on *compute_bound* at the default price the cheapest fixed option (97.6%) beats every adaptive variant. The onboard price takes that scenario's decision-caused battery breaches from 454 (policy alone) and 154 (full stack) to 0.

The system M4 runs (Table V: M3 menu, current price rule, conformal cost feedback) holds 99.3–99.6% where deadlines bind, against 96.4–96.5% for the best fixed option, with no battery or thermal breach on any scenario or on ten further development seeds each. Against the ceiling matched to the filter, *draining*'s 64.6% is 94.1%. It stays below the cheapest fixed option on *thermal_ramp* on every seed, because thermal recovery skips, and on *compute_bound* once in ten seeds, by 0.23 points. A pre-declared variant that keeps running the cheapest safe option under heat recovers 83.2% against 64.9% there but causes one decision-caused thermal breach.

I. Held-Out Evaluation and Phase 3 Status

Outside the board sessions, controller results are simulated over host-measured costs and board energy is estimated. Phase 3 closes that in order (Table XV, Appendix J): board energy once the meter arrives, all ten classes on the board, the M3 menu re-profiled there, and power state as an action.

Held-out evaluation. Phase 3 ends with one evaluation, M4, on ten held-out seeds and two held-out scenarios, run once with thresholds and prices frozen and reported whichever way it falls. Its rules were committed before any held-out run, in two stages: how every setting is derived (2 October), then the values (by 8 October). Its predictions: no decision-caused breach; the onboard price within two points of the hindsight price; quality at least that of the cheapest fixed option; power state tested as an action if the meter arrives; and *the thermal variant adopted only if it gains quality on the held-out thermal scenario with no more decision-caused thermal breaches*. Development data already says where they are weakest (Table V): *thermal_ramp* on every seed, *compute_bound* once, and a thermal variant the test would reject. The threats to validity we know of — a loose ceiling where the filter forbids frames whatever runs, declared survival thresholds, a placeholder value detector, a board that is not radiation-tolerant — are in Appendix J.

V. CONCLUSION

Adapting the model saves about half the energy per inference on a saturated accelerator and nothing measurable at the frame rates a satellite payload uses, because the platform's fixed draw dominates each paced frame: on a duty-cycled payload the energy lever is how often the payload is powered, not how deep the network runs. What adapting buys is deadline safety, and only with a cost model measured on the machine that runs it: on our Raspberry Pi payload a host-profiled menu was 6.5× optimistic, and correcting it online recovered a third of the latency and half the missed deadlines. The menu, not the controller, limits what adaptation can win. The next milestones (Table XV) are measured board energy, a wider certified menu, the payload's power state as a controller action, and one frozen evaluation on scenarios the controller has never seen.

ARTIFACT AVAILABILITY

Code, configurations, raw result folders and a claim checker that recomputes every registered number in this paper from its artefact: https://github.com/Ahmed-Mnasri1/INTERSTELLAR, public from the final submission. The appendices after the references give the detail behind each section.

REFERENCES

[1] N. Fernandez, I. Valdes, S. Van Vaerenbergh, I. de la Iglesia, and J. Arratibel, "A comparative study of CNN optimization methods for edge AI: Exploring the role of early exits," *arXiv preprint arXiv:2604.14789*, 2026.

[2] Y. Zhao, S. Li, and X. Feng, "Lightweight remote sensing scene classification on edge devices via knowledge distillation and early-exit," in *Proceedings of the 33rd ACM International Conference on Multimedia (MM)*, 2025.

[3] Tiansuan Constellation, "Tiansuan experiment platform: Specifications of the BUPT-1 satellite," GitHub repository TiansuanConstellation/TiansuanExperimentPlatform, [Online]. Available: https://github.com/TiansuanConstellation/TiansuanExperimentPlatform. Accessed: Sep. 21, 2026.
[4] C. Wang, Y. Zhang, Q. Li, A. Zhou, and S. Wang, "Satellite computing: A case study of cloud-native satellites," *arXiv preprint arXiv:2307.08530*, 2023.
[5] R. Xing, M. Xu, A. Zhou, Q. Li, Y. Zhang, F. Qian, and S. Wang, "Deciphering the enigma of satellite computing with COTS devices: Measurement and analysis," in *Proc. ACM International Conference on Mobile Computing and Networking (MobiCom)*, 2024.
[6] B. Agüera y Arcas, T. Beals, M. Biggs, J. V. Bloom, T. Fischbacher, K. Gromov, U. Köster, R. Pravahan, and J. Manyika, "Towards a future space-based, highly scalable AI infrastructure system design," *arXiv preprint arXiv:2511.19468*, 2025.
[7] R. Thummala and G. Falco, "When to compute in space," *arXiv preprint arXiv:2512.17054*, 2025.
[8] G. Giuffrida, L. Fanucci, G. Meoni, M. Batic, L. Buckley, A. Dunne, C. van Dijk, M. Esposito, J. Hefele, N. Vercruyssen, G. Furano, M. Pastena, and J. Aschbacher, "The Φ-Sat-1 mission: The first on-board deep neural network demonstrator for satellite earth observation," *IEEE Transactions on Geoscience and Remote Sensing*, 2022.
[9] A. Duggan, B. Andrade, and H. Afli, "Advancing earth observation: A survey on AI-powered image processing in satellites," *arXiv preprint arXiv:2501.12030*, 2025.
[10] C. Wu, Y. Li, M. Xu, C. Guo, Z. Yin, W. Gao, and C. Chi, "A comprehensive survey on orbital edge computing: Systems, applications, and algorithms," *arXiv preprint arXiv:2306.00275*, 2023.
[11] B. Denby and B. Lucia, "Orbital edge computing: Nanosatellite constellations as a new class of computer system," in *Proc. 25th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.
[12] B. Denby, K. Chintalapudi, R. Chandra, B. Lucia, and S. Noghabi, "Kodan: Addressing the computational bottleneck in space," in *Proc. 28th ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), Vol. 3*, 2023, pp. 392–403.
[13] A. K. Erol and D. Mahajan, "Equinox: Decentralized scheduling for hardware-aware orbital intelligence," *arXiv preprint arXiv:2604.19958*, 2026.
[14] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in *Proc. International Conference on Pattern Recognition (ICPR)*, 2016.
[15] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Q. Weinberger, "Multi-scale dense networks for resource efficient image classification," in *Proc. International Conference on Learning Representations (ICLR)*, 2018.
[16] Y. Kaya, S. Hong, and T. Dumitras, "Shallow-deep networks: Understanding and mitigating network overthinking," in *Proc. International Conference on Machine Learning (ICML)*, 2019.
[17] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, and Y. Wang, "Dynamic neural networks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, 2022.
[18] B. Fang, X. Zeng, and M. Zhang, "NestDNN: Resource-aware multi-tenant on-device deep learning for continuous mobile vision," in *Proc. 24th Annual Int. Conf. Mobile Computing and Networking (MobiCom)*, 2018.
[19] Z. Zhang, Y. Zhao, M.-C. Chang, C. Lin, and J. Liu, "E4: Energy-efficient DNN inference for edge video analytics via early-exit and DVFS," *arXiv preprint arXiv:2503.04865*, 2025.
[20] Y. Xue and J. Huang, "ReGate: Enabling power gating in neural processing units," in *Proc. 58th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2025.
[21] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. International Conference on Machine Learning (ICML)*, 2017.
[22] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, and L. Lei, "Learn then test: Calibrating predictive algorithms to achieve risk control," *arXiv preprint arXiv:2110.01052*, 2021.
[23] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
[24] L. Sha, "Using simplicity to control complexity," *IEEE Software*, vol. 18, no. 4, pp. 20–28, 2001.
[25] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conference on Artificial Intelligence*, 2018.
[26] K. P. Wabersich and M. N. Zeilinger, "A predictive safety filter for learning-based control of constrained nonlinear dynamical systems," *Automatica*, vol. 129, p. 109597, 2021.
[27] I. Gibbs and E. Candès, "Adaptive conformal inference under distribution shift," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
[28] A. Badanidiyuru, R. Kleinberg, and A. Slivkins, "Bandits with knapsacks," *Journal of the ACM*, vol. 65, no. 3, pp. 13:1–13:55, 2018.
[29] L. Huang and M. J. Neely, "Utility optimal scheduling in energy-harvesting networks," *IEEE/ACM Transactions on Networking*, vol. 21, no. 4, pp. 1117–1130, 2013.
[30] P. Helber, B. Bischke, A. Dengel, and D. Borth, "EuroSAT: A novel dataset and deep learning benchmark for land use and land cover classification," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 12, no. 7, pp. 2217–2226, 2019.
[31] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
[32] T. Pfandzelter, N. Bauer, A. Leis, C. Perdrizet, F. Trautwein, T. Schirmer, O. Abboud, and D. Bermbach, "Trabant: A serverless architecture for multi-tenant orbital edge computing," *arXiv preprint arXiv:2504.08337*, 2025.
[33] Z. Wang, "Space AI: Leveraging artificial intelligence for space to improve life on earth," *arXiv preprint arXiv:2512.22399*, 2025.
[34] P. Antunes and A. Podobas, "FPGA-based neural network accelerators for space applications: A survey," *arXiv preprint arXiv:2504.16173*, 2025.
[35] D. N. Singh, "Which workloads belong in orbit? A workload-first framework for orbital data centers using semantic abstraction," *arXiv preprint arXiv:2603.20317*, 2026.
[36] L. He and J. Xu, "SAT-Edge-Agent: Hardware-in-the-loop edge-agent orchestration for onboard satellite intelligence," arXiv preprint arXiv:2608.03728, 2026.
[37] Y. Matsubara, M. Levorato, and F. Restuccia, "Split computing and early exiting for deep learning applications: Survey and research challenges," *ACM Computing Surveys*, vol. 55, no. 5, pp. 1–30, 2022.
[38] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.

Appendix A
Related Work in Full

Section II summarises each strand in a sentence or two. This appendix gives them as written, the comparison table the body refers to, and the gap stated in full.

Onboard inference and orbital compute. Running inference on the satellite rather than downlinking raw data is established practice: Φ-Sat-1 flew a convolutional network on a Myriad 2 vision processor to discard cloudy tiles before transmission [8], and surveys map onboard image processing [9] and orbital edge computing [10]. Orbital edge computing recasts a constellation as a computer that adapts to orbit and power conditions, evaluated in simulation [11]; Kodan selects specialised models per sample so that every tile fits the frame deadline, and measures time but not energy [12]. Infrastructure work spans multi-tenant platforms [32], cloud-native satellites [4], orbital data centres [6], [33] and radiation-tolerant accelerators [34]. In-orbit measurement of COTS computers aboard BUPT-1 finds temperature and energy, not compute, to be the binding constraints [5]. A parallel strand decides *where* a workload should run [7], [35]. Agentic orchestration

on flight-class ARM boards has begun to appear: SAT-Edge-Agent runs a detector and a language service behind one mission interface on a COTS ARM system-on-chip, finds the detector to be under 3% of end-to-end latency, and reports board power only from an unsynchronised plug-meter pilot [36]. We ask the next question: once a workload is onboard, how should it execute as its budgets move?

Dynamic and early-exit inference. BranchyNet exits at early classifiers when confidence is high [14], MSDNet targets anytime prediction under a budget [15], and shallow-deep networks show that some inputs are right early and wrong at full depth [16]; the field is surveyed in [17], [37], and efficient backbones such as MobileNetV3 [31] compose with it. These works adapt to the *input*, not to the platform, and report FLOPs or latency. The most recent comparison of early exits on a Jetson Orin and a Raspberry Pi evaluates accuracy, latency and model size, without measuring energy at all [1]. Closest to our payload, E3C distils a remote-sensing scene classifier with early exits and meters it on a Jetson AGX Orin, an Orin Nano and a Raspberry Pi 4B with an external USB power meter, reporting energy-efficiency gains of over 40% [2]. Its exits still adapt to input confidence alone, and its energy is reported as an efficiency gain rather than separated into the board's fixed draw and the share a policy controls.

Resource-aware runtime adaptation. NestDNN resizes a nested model to the resources available on a smartphone and reports the energy saved [18]; E4 couples exit selection with per-layer DVFS on five Jetson boards and measures energy with on-board sensors [19]. These systems choose how much to compute. None separates the platform's fixed draw from the energy that choice controls, and none bounds what its controller may do when the cost model is wrong.

Value-priced scheduling in orbit. Closest to our controller in spirit, Equinox compresses battery charge, temperature and queue backlog into one multiplicative, barrier-shaped marginal cost, runs an image's task only when its expected science value exceeds that cost, and offloads to a neighbouring satellite whose cost is lower [13]. Lower-value tasks are therefore shed first as the battery falls. It decides *which* work runs and *where*, across a simulated 143-satellite constellation; its coefficients are calibrated by hand, its limits enter as costs rather than as a veto, and its task costs are fixed from an offline profile of a Jetson board. Our rule (7) reduces to its admission test for a one-option menu. MIZAN instead chooses *how* to run among measured options, derives its price from the energy budget without a tuning constant, enforces limits with a filter that can only veto, and corrects its costs on the board. Across a constellation, the price itself is the scalar satellites would exchange to offload work, as Equinox does with its cost. Pricing stored energy is older than both: energy-harvesting schedulers price it through a Lyapunov virtual queue [29].

Where the energy goes. Measuring neural processing units directly, ReGate attributes 30–72% of energy to static power and a further 17–32% to idleness, and recovers it by power-gating idle components [20]. Energy spent because a chip is on, rather than because it computes, is invisible to any policy that only chooses how much to compute. We find the same structure one level up, at the board. The remaining lever, processing quality, is most directly reached through integer quantization [38]; on our ARM target that is left for future work.

Confidence and risk. Gating needs calibrated confidence; temperature scaling is the standard correction [21]. Selective classification frames abstention as a risk–coverage trade-off [23], and Learn then Test certifies a configuration with a finite-sample risk guarantee, provided the certifying data did not choose it [22]. Our first guarantee broke that condition; we report the corrected protocol.

Safety filters and budgets. A layer that may veto an optimising or learned controller but never choose for it is an established construct: the Simplex architecture switches to a verified fallback [24], shields block a learning agent's unsafe actions [25], and predictive safety filters minimally modify a proposed input to keep a constrained system recoverable [26]. Ours is the simplest of these, a per-frame feasibility test. What we add is where its numbers come from — costs measured on the target and corrected online with a distribution-free latency bound whose miss rate adapts under drift [27] — and an energy price for the policy it guards. That price is the Lagrange multiplier of a budgeted sequential decision, the setting bandits with knapsacks study when rewards are unknown [28]; here the menu states them, so the multiplier has a closed form.

The gap, and what we add. The gap is not that nobody has run an early-exit network onboard; Table VII shows it is narrower and of four kinds. *An unexplored condition:* one row meters the same board class, but no row measures what adaptation is worth in energy on a *duty-cycled* flight-class board, where the fixed draw is most of the bill. *A trade-off:* gated inference is usually presented as free, and our retraining finds it costs 1.70 ± 0.38 pp at 64 pixels. *A methodological gap:* costs are profiled on one host and trusted on another, and board energy is reported without idle subtraction; both fail measurably here (Sections IV-B and IV-F). *An integration gap:* elastic inference, resource-aware control and platform energy accounting are mature apart, and no row closes them into one loop whose decisions use the costs that were measured, under a filter that can only veto. The last row of the table answers the last three in full; the first it answers on three accelerators, and partially on the target board.

Appendix B
Methodology in Full

Section III keeps every equation with the prose needed to read it. This appendix gives that section's text in full, in its order: the margins, the derivations, the rule histories and the metric definitions the body leaves out.

A. Problem Formulation

The menu and its costs. At every frame the onboard system either skips the observation or runs one configuration from a menu of options profiled offline. When energy has not been measured on the target device, it is estimated as $P_{\text{act}}\,\bar{\ell}(a)$ from

TABLE VII

REPRESENTATIVE PRIOR WORK AGAINST THE PROPERTIES THAT MATTER HERE. EACH ENTRY SUMMARISES WHAT THE CITED WORK ITSELF REPORTS; "—" MEANS THE WORK DOES NOT ADDRESS THAT PROPERTY.

Work	Platform evaluated	Adapts at runtime to	Energy evidence	Stated guarantee
Φ-Sat-1 [8]	Myriad 2 VPU, in orbit	— (fixed cloud filter)	payload-level power only	—
COTS in orbit [5]	COTS boards aboard BUPT-1	— (measurement study)	in-orbit platform energy and thermal	—
OEC [11]	simulation	orbit and power conditions	simulated	—
Kodan [12]	Jetson AGX Orin, CPU, GPU	per-frame compute deadline	none (time only)	—
Early exit [14], [15], [16]	host CPU or GPU	input difficulty only	FLOPs or latency	—
NestDNN [18]	smartphones	available resources	reported	—
E4 [19]	five Jetson boards	frame complexity, with DVFS	on-board sensors	—
Edge early exit [1]	Jetson Orin, Raspberry Pi	— (static comparison)	not measured	—
E3C [2]	Jetson AGX Orin, Orin Nano, Raspberry Pi 4B	input confidence only	external meter; efficiency gain, fixed draw not separated	—
Equinox [13]	simulated 143-satellite constellation; latency profiled on a Jetson Orin Nano Super	battery, temperature, queue and task value (admit, defer, offload)	simulated; board idle trace	value-ordered task dropout (proof)
MIZAN (ours)	Raspberry Pi 4B (target); A40, L4, T4	deadline, energy, memory and temperature, in closed loop	GPU boards, total and above idle, 95% intervals; target board pending meter	selective risk (certified rung); latency bound with stated miss rate; veto-only filter

a declared active power and labelled as an estimate in every result. Our frozen menu's eight options are fixed exits 1–3 at 48 or 64 pixels and two confidence-gated adaptive policies, with accuracy measured on 2,000 independent EuroSAT images. Section IV-F measures how far those costs transfer to the target board, and the answer bears directly on the margins below. Phase 3, and the held-out evaluation M4, run its certified five-rung successor (Table I).

State and charges. The controller observes the energy stored in the battery, the processor temperature, the free memory and the time budget available to inference, together with the limits $b_{\min}$ and $\theta_{\max}$. In (1) and (2), $a_{t'}$ is the last executed option, and the switching costs $\ell_{\text{sw}}, E_{\text{sw}}$ come from measured transitions. ℓ_t is the latency the filter plans with; the board results of Sections IV-F and IV-G count a miss when the latency a frame actually took exceeds its deadline. A skipped frame pays only $P_{\text{act}}\,\tau_t$. The platform evolves with the orbit through (3) and (4).

What the objective can express. Here $v_t \in [0, 1]$ is what frame t is worth to the mission. Every result is scored with the true value, not the detector's estimate, so a controller cannot score well by being wrong in its own favour. The sum in (5) is therefore the value-weighted number of correctly processed frames, and a skipped or vetoed frame counts as an error. Constraints are never traded against accuracy: a solution that violates one is infeasible, not penalised. $Q(a)$ is the dataset-level accuracy of a configuration, not an estimate for the current input, which limits how much input-aware adaptation the formulation can express.

Scope of the action space. Two levers act through this formulation and one does not. The menu $\mathcal{A}$ carries the model and processing-quality levers (exit depth and input resolution), and SKIP carries the frequency lever at the granularity of a frame. What SKIP does *not* do is power the payload down: the board's fixed draw is folded into P_{hk} in (3) and charged on every frame, executed or not, so it lies outside what the controller decides. We state this boundary deliberately, because Section IV finds that fixed draw, not network depth, dominates a paced frame's energy. The per-frame decision therefore governs deadline and accuracy within an awake window; turning the payload's own power state into an action, with a sleep power, a wake-up cost and a wake-up latency, is the first extension of the Phase 3 plan (Appendix J).

B. Controller Architecture

The loop. The controller proposes, a deterministic safety filter disposes, and the executor runs the approved option on the inference engine. The controller costs $O(K)$ per frame and runs unchanged on the target board, which Section IV-F confirms by running it there. In Fig. 2 the environment produces the resource state and the detector's estimate of the frame's value; the executor runs the approved option, and the frame's charged cost updates the next state; energy on the board is the declared active power over measured latency. The harness logs every frame.

Safety filter. The filter approves a proposal only if it belongs to (6); otherwise the frame becomes SKIP. The margins reserve overhead that is only known after the decision: $\delta_\ell = 2\,\text{ms}$ covers the measured switching overhead (1.25 ms, the largest mean over 56 measured transitions) plus the controller's decision time, and the harness counts every frame where the actual overhead exceeded a margin. The test is per frame and uses ℓ_{95}, so it is conservative for latency but does not look ahead in time. Two results bear on this margin. It covers every controller without cost feedback on ARM, but not the feedback layer: on the board that took up to $1.19\,\text{ms}$ per decision, and $1.25{+}1.19\,\text{ms}$ exceeds it, which happened on 30 of 360 frames at the board's low clock (Section IV-F). And it is stated against menu latencies that the target board does not reproduce.

Pareto policy. In (7), $Q(\varnothing) = E(\varnothing) = 0$, so SKIP scores zero and ties are broken towards lower energy. SKIP is part of the argmax, so an option runs only when it is worth its energy at price λ. With our menu $E(a) \le 12.4\,\text{mJ}$, so at $\lambda = 1\,\text{J}^{-1}$

on a frame valued at 1 the energy term only separates options whose accuracy differs by less than about one point, and the rule otherwise selects the most accurate feasible option.

Value-ordered shedding. Since SKIP scores zero in (7), a frame runs only while $\lambda < \hat{v}_t \rho$, with $\rho = \max_{a \in \mathcal{A}_{\text{safe}}} Q(a)/E(a)$: the price that sheds a frame grows linearly with its estimated value, so an off-target frame ($\hat{v} = 0.1$) is shed at a tenth of the price that sheds an on-target one, and because the chosen rung depends on $\lambda/\hat{v}_t$, a higher-value frame also keeps a richer rung longer. This is the value-ordered dropout that Equinox proves for its barrier cost [13]; under (7) it holds for any price.

Pricing energy without hindsight. λ is the Lagrange multiplier on the energy budget of the offline optimum (11), so it is the marginal accuracy a joule buys, and its value decides which rung of the menu the argmax settles on. Tuning it per trajectory needs foresight of the whole episode, which no onboard controller has. It does not have to be tuned, because the menu states it. In (8), T_t is what each remaining frame may spend if the charge is to cover the rest of the episode once the platform has taken its share. When $(H-t)\,d$ exceeds the usable charge, the floor will be lost whatever the controller does, and the price drops to its minimum. A spacecraft knows its episode length and its orbit-average power budget from mission design, so both are declared mission parameters, not tuning constants. The interim version of this rule measured the draw from the battery and divided by the whole horizon on every frame; Section IV-D reports both, and Appendix J why the rule changed. The price (9) is taken as the previous value when no richer rung exists, which is the case whenever the allowance already covers the most accurate option. Both are $O(K)$ over the pruned menu and are recomputed every frame, so the price tracks a falling charge instead of being approached by a gradient.

Hysteresis. A wrapper holds the current option for at least $D = 3$ frames to avoid paying switching costs on every fluctuation. The hold is released at once when the held option becomes infeasible, so hysteresis never delays a safety-driven change.

Cost feedback. The menu is a prediction made on a development host, and the board runs it about $6.5\times$ slower (Section IV-F), so the filter's latency test is only as honest as the menu. An outer wrapper therefore hands every layer below it the menu at its currently estimated cost. The bound (10) is the k-th smallest of the last W latencies. Adaptive conformal inference keeps the long-run share of frames above their bound near δ under drift. The window's mean replaces the mean that energy is priced on. The interim board sessions ran an earlier rule, an exponential average of observed over predicted latency; Section IV-G compares the two on the board's own latencies.

Survival mode. The outer layer is a finite-state machine over usable energy and temperature. NOMINAL passes the full menu; SAVING restricts it to the cheaper half; CRITICAL to the cheapest option; EMERGENCY switches inference off without calling the inner layers; THERMAL_RECOVERY skips frames from $2\,^{\circ}\text{C}$ below the limit until the processor is $5\,^{\circ}\text{C}$ below it. Energy thresholds are expressed as time to the battery floor at the unavoidable platform draw rather than as bare joules: SAVING below 10 minutes of reserve, CRITICAL below 4 minutes, and a one-minute band before any return towards NOMINAL, so the mode cannot oscillate at a boundary. Each threshold is capped at a declared share of the reserve the episode starts with (50, 20 and 10%), because a horizon longer than the reserve would place the whole episode below SAVING and throttle every controller identically; the cap binds only in *compute_bound*.

Overhead. Controller decision time is charged in every experiment through (1) and (2). On ARM it depends on the layers: 0.02–0.03 ms for a fixed option, 0.08–0.13 ms for the adaptive policy, and 0.35–0.55 ms with cost feedback (run means, Section IV-F).

C. Closed-Loop Evaluation Protocol

References and bounds. Controller time is charged as a declared constant (0.02 ms) for exact repeatability, while the measured time is logged. The best static option chosen with hindsight of the trajectory is obtained by running every option. An upper bound assigns each frame its most accurate option that fits the frame's deadline and memory, ignoring energy and heat. That per-frame bound ignores every consequence of spending, so it cannot bound a controller that must also keep the platform alive. For the denominator we therefore compute the offline optimum (11) for each trajectory. Over one episode the energy available to inference is fixed: the charge above the floor, plus generation, less the housekeeping and detection the platform draws whatever is decided. Each frame is then decided on its own at a price λ per joule. Bisection on λ drives spending onto the budget, where the bound is tightest. The same price yields an achievable policy, which we replay through the environment's own transition function to score it exactly; the bound and that replay bracket the true optimum, and we report the gap rather than assuming it small.

The bound is deliberately optimistic — full foresight, no switching cost, only the episode's total energy constrained — which is the safe direction for a denominator. It is tight on five of seven scenarios; on the other two its own policy breaks a hard limit, and we say so wherever the figure is quoted. Ablations remove one component at a time while the safety filter stays on.

The ceiling matched to the safety filter. Phase 3 adds a second ceiling, matched to the safety filter. Every policy pays at least a skip's energy on every frame, and both the battery and the thermal update are monotone in what is spent, so on every frame each policy's charge is at most, and its temperature at least, those of the idle trajectory; every test in (6) is monotone the same way. Offering each frame only the options the filter would approve on that idle trajectory therefore gives a superset of what it approves under any policy, and (11) over that superset still bounds every controller the filter governs. It differs from the first ceiling only where the filter forbids frames whatever runs: on *draining* and *thermal_ramp*

the matched ceiling is 68.6% and 84.8% of the unmatched one (M3 menu); the 2 ms latency margin alone takes under a point on the two deadline scenarios, and elsewhere the two coincide. Every share in this paper is of the unmatched ceiling unless its caption says otherwise.

Metrics. Metrics follow the five indicators. Accuracy is reported per frame, with skipped and vetoed frames counted as errors. Inference time is the 95th-percentile frame latency including controller and switching time. Consumption is the sum of inference, controller and switching energy, measured where a meter is available and otherwise estimated and labelled as such; platform draws come from the simulator. Memory is the resident memory of the executed option. Robustness, as the held-out evaluation defines it, is true value-weighted quality as a share of the offline optimum while the available resources vary; Section IV-E also reports accuracy under a varying time budget. Constraint violations are reported as counts, never averaged, and split into unavoidable violations, those of a reference that never runs inference on the same trajectory, and violations caused by decisions. Finally, the net energy gain

$$G = \frac{E_{\text{inf}}^{\text{ref}} - E_{\text{inf}}}{O - O^{\text{ref}}}, \qquad O = E_{\text{ctrl}} + E_{\text{sw}}, \tag{13}$$

compares the inference energy an adaptive controller saves with the extra overhead it pays against a static reference; $G > 1$ holds exactly when the adaptive controller uses less total energy, and G is always read next to accuracy.

D. Adaptive Inference Model and Its Menu

Accuracy is not the target. The controller can only choose among options the inference engine can execute, at costs that were measured. Our small network reaches 95.8% at full depth on a random split (the dataset's authors report up to 98.57% with a full-size network); most menu points sit at 85–91%, chosen for cost. We do not compete on accuracy: the menu answers what each operating point costs on the hardware that runs it.

One network, many operating points. All heads and weights stay resident, so early exit reduces executed work, not stored parameters; resident memory is effectively the same across every option and is not a discriminating resource in this menu.

Confidence gating. Each head is temperature-scaled on a calibration split that the training never saw. At inference, an image that does not pass a head's threshold descends to the next. Because later heads reuse the features an earlier head already computed, a gated run costs exactly the path it took and nothing more; we verified this equivalence exactly rather than assuming it, by routing recorded fixed-exit outputs through the gates and reproducing the executed adaptive run on thousands of evaluated images with zero mismatches in exit, class or confidence. A depth cap is exposed as a separate knob: it bounds the worst-case path for deadline-critical frames without changing the gates themselves.

Thresholds with a stated guarantee. Choosing λ_k by cheapest-cost-within-tolerance on development data transfers badly: on a geographically held-out test region such a policy lost 2.28 pp of accuracy against full depth ($p = 0.001$). We therefore select thresholds by risk certification instead. We use $\alpha = \delta = 0.10$, split over the two resolutions, and state plainly what the statement does not cover: it is conditional on early exit, not on all frames; it assumes exchangeability, which a geographically distinct test region does not strictly satisfy; and it says nothing about latency or energy.

TABLE VIII

THE FROZEN MENU. ACCURACY ON 2,000 HELD-OUT IMAGES THAT NEITHER TRAINED THE MODEL NOR SELECTED THE THRESHOLDS; LATENCY AND MEMORY FROM ISOLATED SINGLE-IMAGE CPU PROCESSES ON THE DEVELOPMENT HOST. SECTION IV-F MEASURES HOW FAR THESE COSTS TRANSFER TO THE TARGET BOARD. ONE TRAINING RUN: ITS ADAPTIVE OPTION ABOVE FULL DEPTH AT 64 PIXELS, AND ITS WEAK 48-PIXEL FULL DEPTH, DID NOT SURVIVE RETRAINING (BELOW).

Res.	Option	Acc. (%)	Mean (ms)	ℓ_{95} (ms)	RSS (MB)
64	adaptive $\lambda = 0.7$	90.7	2.87	6.24	307
64	full depth	89.2	6.20	9.61	315
64	exit 2	88.8	4.44	6.64	306
64	exit 1	86.5	1.76	2.78	315
48	adaptive $\lambda = 0.7$	86.8	3.18	7.11	312
48	exit 2	85.3	4.59	7.15	311
48	exit 1	85.2	2.26	3.25	311
48	full depth	69.3	6.13	8.75	312

Costs are measured, and refused when not measured. Every option in the menu carries accuracy from images that neither trained the model nor selected its thresholds, and latency, memory and energy measured by running that exact checkpoint. A configuration that has never actually been executed cannot enter the menu on an interpolated guess. Table VIII lists the eight frozen options the controller uses in this paper.

What retraining changed. The frozen menu comes from one training run whose 48-pixel final head was unusually weak, which flattered gating. A pre-registered retraining over five seeds, on random and geographic splits, found three things: adaptive inference costs about 1.7 points at 64 pixels (a trade-off, not a free gain); training at both resolutions repairs the 48-pixel head; and a geographic split costs about five points. The certified successor, evaluated once on the held-out region, reaches 90.15% against 91.02% at full depth (not significant) at under half the mean latency. Results in Section IV use the frozen menu unless they say otherwise; Phase 3 and the held-out evaluation run five rungs of the successor (Table I). Its costs come from isolated CPU processes on the development host (two threads). Only the certified rung's thresholds come from the certification procedure, so no accuracy in that table is a certified bound. Both depth-capped rungs lie below the frontier a linear energy price can reach, so they run only when a deadline excludes the rungs on it.

APPENDIX C
THE OFFLINE OPTIMUM

Section III-C defines the offline optimum used as the denominator throughout, and states that it is tight on five of seven scenarios. This appendix gives the two things that

statement rests on: why the bound is computed the way it is, and what it does on the other two scenarios. It also records the method we tried first for the energy price and abandoned.

Why not a grid dynamic program. A grid dynamic program was tried first and abandoned: one frame moves the battery by a few millijoules against grid cells of a fraction of a joule, so discretisation swamped the quantity being bounded.

Where the bound is not tight, quantified. The bound is deliberately optimistic, which is the safe direction for a denominator: it has full foresight, pays no switching cost, and constrains only the episode's total energy, so a policy that would overheat or dip below the floor mid-episode is still counted. That matters unevenly across scenarios, and we quantify it rather than leaving it qualitative. Replaying the bound's own policy shows it respects both hard limits on five of our seven scenarios, where the ceiling is therefore tight; on *draining* it breaches the battery floor on 101 of 400 frames, and on *thermal_ramp* it exceeds the thermal limit on 259 of 1,116. On those two the ceiling is not a policy any controller could run, so a shortfall against it is not evidence of a control failure, and we say so wherever the figure is quoted. These counts are regenerated by `experiments/price.py` alongside the results of Section IV-D.

Dual ascent, tried first and worse than doing nothing. Dual ascent on λ — the textbook way to find a multiplier, and the one we implemented first — is *worse* than the closed form and worse than the untuned default: the multiplicative step compounds while its own target shrinks, driving the price to 802–8233 and skipping most of the episode. With (9) in place the result is insensitive to the step size, which is to say the gradient contributes nothing.

Appendix D
Controller Overhead on ARM

A controller that decides every frame must cost far less than the frame it decides about. This appendix reports what it costs, on the development host and on the target board, and investigates the one variant whose isolated maxima exceed the latency margin.

Overhead. On the development host the decision time of every controller, safety filter included, has a median of 0.003–0.012 ms and a 99.9th percentile below 0.1 ms over 60,000 decisions each; the median is under 1% of the fastest option's mean latency. On the target board the same benchmark gives a median of 0.009–0.029 ms, a factor of 3.7–4.8 slower, with a 99.9th percentile that never exceeds 0.131 ms over thirteen runs — about $15\times$ inside the $\delta_\ell = 2$ ms margin. Isolated maxima of 2.3–5.7 ms do occur, and they exceed the margin: δ_ℓ is sized on the 99.9th percentile and is explicitly not a worst-case bound, which is the same distinction we insist on for every observed percentile in this paper. Those maxima are not scattered noise. They appear in the survival controller in every run, and in whichever controller is measured immediately after it. The overhead is charged in every experiment through (1) and (2).

TABLE IX
Controller decision time (ms) on ARM against the development host, same menu, 60,000 samples per controller. Pi and p99.9 columns are from the second session; the maximum column gives the range over the second and third sessions, the third measured in both declared and reversed order.

Controller	Host	Pi	p99.9	max
static	0.0019	0.0091	0.032	0.09–0.35
rule-based	0.0023	0.0106	0.034	0.09–0.35
Pareto	0.0057	0.0216	0.082	0.18–0.39
+ hysteresis	0.0059	0.0222	0.093	0.16–0.41
+ survival	0.0079	0.0292	0.116	2.51–3.11

The controller costs 0.03 ms on ARM, and its survival mode carries an attributable allocation pause. Table IX reports decision time over 60,000 samples per controller, standard library only, with no torch, no images and no checkpoint loaded. ARM is a consistent 3.7–4.8$\times$ slower than the host, but the 99.9th percentile never exceeds 0.131 ms in any of the thirteen committed runs (0.116 ms in the second session, the one tabulated), about $15\times$ under the 2 ms margin of (6): against a $\sim$22 ms inference the controller costs 0.13% of the frame, which is what makes a per-frame decision affordable at all on this class of hardware.

The margin is sized on the 99.9th percentile and covers it comfortably; it does not cover the observed maximum, and we state that rather than leave it to be discovered. `survival(pareto:1+hysteresis3)` reached 2.5–3.1 ms in every run of the second and third sessions, reproducibly above the margin. Running the benchmark in reversed order identifies the cause: the spike moves with survival from the last slot to the first, so it follows the controller and not the position, and `pareto:1+hysteresis3` — clean in declared order — itself spikes to 2.33–2.40 ms when reversal places it *immediately after* survival. The pause therefore belongs to survival's allocation behaviour and lands on survival or on whichever controller follows it, the signature of a generational collector firing on allocation counts rather than wall-clock time. It is neither an operating-system deschedule nor the cost of deciding, which the same runs measure at 0.12 ms at the 99.9th percentile. Since the mechanism is inferred from the ordering experiment rather than from a profiler, and the magnitude is not constant across sessions (4.65–5.72 ms in the first), we report an attributable runtime pause with a known trigger rather than as a solved defect. A single 38 ms outlier in the first session did not recur and is treated as a genuine system stall.

Appendix E
Experimental Setup, Provenance and the Simulated Environment

Section IV-A states the platforms in one paragraph. This appendix gives them in full, the provenance of every quantity (Table X), and the simulated environment. Results come from four places, and the paper is only worth as much as the boundary between them is clear.

Why these platforms. The Raspberry Pi 4B is not a stand-in for flight hardware but the flight hardware of one research mission: the BUPT-1 CubeSat of the Tiansuan constellation carries two of exactly this board as its general-purpose computing payload [3], [4]. The three data-centre accelerators are not a second target but a contrasting regime: they share the hardware *class* that recent orbital data-centre proposals assume [6], [7], and it is where the model lever behaves entirely differently. They do not share the utilisation those proposals assume: at batch one, a MobileNetV3-Small does not come close to filling an A40, so what we measure is the accelerator's behaviour on a small network, not a data centre's workload.

Evidence behind Fig. 1. Deadline safety: Sections IV-E and IV-G. The two accelerator cells: Section IV-B and Table III. The platform lever: Sections IV-B and IV-F. Each cell's badge uses the provenance labels of Table X.

Target board. A Raspberry Pi 4 Model B (8 GB RAM), 64-bit Linux, the same pinned software the menu was profiled with; inference is ARM CPU work. The deployed model was verified identical to the host copy and the full test suite passes on the board. The first three board sessions ran with the desktop active on synthetic images, so their absolute latencies are upper bounds, except the uncontended floor, which reproduces across sessions. The fourth ran headless on real EuroSAT images, with the cheapest fixed option interleaved with both cost-feedback arms (Section IV-F).

Development host. A laptop CPU under the same pinned software. It profiled the frozen menu of Table VIII, so it is the subject rather than the authority of Section IV-F.

Accelerators. An NVIDIA A40 and cloud L4 and T4 instances, each with its own power telemetry. They are not a deployment target but the data-centre hardware class that orbital data-centre proposals assume, measured at batch size one, far below the occupancy such proposals plan for.

Task and data. EuroSAT RGB land-use classification [30], split into disjoint groups for training, threshold selection, certification and final evaluation, with 2,000 images reserved for the independent accuracy of every menu entry; no split is used to evaluate a model it helped select. A random split is not a geographic split: our retraining study measures about five accuracy points of difference between them, so the successor menu is evaluated geographically.

Simulator. The closed-loop environment models a battery with a floor, a first-order thermal relaxation, an orbit and duty-cycle model that gates solar generation through eclipse, and a workload model, with a 5 s frame period in most scenarios. Housekeeping power is charged on every frame, including skipped ones, which is what makes skipping a real choice. The results below use seven named scenarios, each built so a stated resource is the one that binds; Phase 3 adds an eighth, *energy_trade*, whose optimum moves between three rungs, and two held-out scenarios no controller has run on (Section IV-I).

Verification. Before any number is reported, we recompute it independently from raw measurements and check that the model checkpoint, calibration data and images used match what was declared; runs whose hardware, precision or measurement source differ are never pooled together.

A. The Simulated Environment: Trajectories, Frontier and Detector Sensitivity

Everything in this subsection is simulated. Energy is integrated from a declared 2 W active power rather than metered, and the detector is a declared component with a declared error rate that nobody has built. Neither may be read as a measurement.

The constraints the scenarios actually impose. Fig. 4 plots, for each of the seven scenarios, how much battery headroom and thermal margin the payload has at every frame, against its own constraint. Two scenarios reach a constraint: *draining* crosses the battery floor, and *thermal_ramp* crosses the temperature limit. The rest never approach one, which is why their controllers coincide — with no binding resource there is nothing to arbitrate, and the three plotted controllers make the same decisions frame for frame. The starting headroom is printed in each panel because it varies by three orders of magnitude: 25.2 kJ in the benign scenarios against 30 J in *compute_bound*, which is what makes that scenario a stress test of the survival thresholds rather than of the menu.

The frontier, scored on truth rather than on the detector's word. Fig. 5 places every controller in energy/quality space per scenario, scoring quality with the environment's true mission value so that a controller cannot score well by being wrong in its own favour. The clustering is the result: in the four scenarios where nothing binds, the full system, the rule-based controller, the Pareto controller, A1 and A3 all land on one point. Separation appears only where a resource binds, and even there it is the input-aware arm (A2) and the static extremes that move, not the layers of the full stack. On *compute_bound* the cheapest static option lies above and to the left of the full controller, more true quality for less energy, which is the finding of Table XII seen from the energy side.

What a worse detector costs. Fig. 6 sweeps the declared detector accuracy in the only two scenarios where the controller skips at all; in the other five it is identical to the no-skip reference, saves no energy, and the ratio that Fig. 6 plots is undefined. On *compute_bound* the trade crosses over between 90% and 95% accuracy: above it, skipping on the detector's word buys both more true quality and less energy, and below it the payload gives up quality to save joules, rising to 53.2 quality points per joule saved at 70%. On *draining* the curve is flat at about 24.5 points per joule across the whole range, because there the skips are driven by the battery rather than by the detector, so the detector's accuracy barely enters. The detector is the largest single assumption in the paper, and this figure is the sensitivity that bounds it, not a result.

A3 is a one-scenario ablation. Removing thermal awareness changes nothing at all on six of the seven scenarios: the full system and A3 produce bit-identical quality, energy and violation counts on *stable*, *orbital*, *draining*, *sudden_drop*, *deadline_pressure* and *compute_bound*. Only *thermal_ramp*

TABLE X

PROVENANCE OF EVERY QUANTITY USED IN THIS PAPER. *MEASURED* MEANS AN INSTRUMENT OR A TIMER OBSERVED IT ON THE STATED PLATFORM; *SIMULATED* MEANS IT WAS INTEGRATED FROM DECLARED ASSUMPTIONS; *ESTIMATED* MEANS IT WAS DERIVED FROM A DECLARED CONSTANT AND CARRIES NO INDEPENDENT INFORMATION.

Quantity	Source	Status
Accuracy per menu option	2,000 images that neither trained nor selected	measured
Latency per option, development host	isolated CPU processes, repeated trials	measured
Latency, target board	Pi 4B, four sessions; synthetic images in three, real in one (Sec. IV-F)	measured
Accuracy, target board	**real images in session 4: every class and exit equals the host's on 2,560 frames; the paced images cover 2 of 10 classes**	**measured, narrow**
Latency bound of the cost-feedback layer	session 4's board latencies replayed in time order	measured, replayed
Controller decision time	tens of thousands of decisions, host and board	measured
Resident memory	sampled process memory, host and board	measured
Energy per inference, three accelerators	on-board power counters; total and above idle	measured
Energy per inference, target board	**no power meter in any of the four sessions**	**not yet available**
Energy shown in board run summaries	declared active power over measured latency	estimated, labelled
Inference's share of a frame's energy, target board	a published measured base draw for this board [32], our declared active increment, our measured latency	estimated
Battery, temperature, eclipse, housekeeping	closed-loop simulator, declared assumptions	simulated
Mission value per frame	1 on target, 0.1 off; seen through a detector right on 90% of frames; results scored on the true value	declared

separates them, and there the separation is large and uncomfortable: A3 scores 30.6% more mission-weighted quality for 17.8% more energy, and incurs 259 thermal violations against the full system's 258. Since 258 of those violations occur in that scenario even when inference never runs — the ambient trajectory causes them, not the controller — the thermal arm's entire measurable benefit is one avoidable violation, bought with 30.6% of the mission value. We report this rather than drop the column: the ablation is honest about a layer that, on the evidence we have, does not yet pay for itself, and a scenario in which the thermal limit is crossed by the workload instead of by the ambient is the experiment that would settle it.

APPENDIX F
RESULTS IN FULL

Section IV states each simulated and accelerator result in a paragraph. This appendix gives those subsections as they stood: setups, readings, limits, and the figure and tables the body refers to. The target board's sessions are in Appendix H.

A. What Adaptive Inference Costs on Real Accelerators (Section IV-B)

We start with what an instrument observed. This subsection measures what adaptive inference costs on real accelerators; Section IV-C then carries those measurements into the closed loop (its right-hand panel).

Protocol. Each option runs alone on an accelerator allocated exclusively to us (requested; other tenants' activity could not be verified, so the idle baseline is measured inside every window), in randomised order, across repeated timed windows, either saturated (as fast as the host can submit work) or paced at a fixed frame rate. Energy comes from each board's own power counter; per-inference energy is the window's counter difference divided by completed inferences. That is the board's total energy, idle draw included, and it is what Table III reports; the savings there are savings of that total, and paced means the board stays powered on between frames. Subtracting an idle-window baseline isolates the energy inference itself adds, E_{dyn} in (12). Every number is checked against raw telemetry before it is reported, and differences between options are reported as confidence intervals over repeated trials, never as single point numbers.

Two regimes, one explanation. The arithmetic is not subtle. Saturated savings are real, and they come from throughput — more completed inferences per joule — not from lower power: active board power differs by under 1 W between options on the T4. Recomputed from each board's raw energy counter, the idle draw is 94.1%, 87.2% and 97.5% of a saturated full-depth inference on the A40, L4 and T4, and 86–97% of the saving is idle power charged over a shorter window; the rest is arithmetic the early exits skip. The consequence for a duty-cycled payload is that adapting the model is not an energy lever; adapting the *frequency*, or powering the accelerator down between bursts, is. We have not demonstrated that second part on the target board itself, and are careful not to claim more there than we have measured: what we do observe on the board is its own frequency governor scaling the processor clock down between paced frames and back up for each inference, which points at the same mechanism from a different angle.

Measurement pitfalls worth naming. Two details changed our numbers during this work and are worth a sentence each, because they are easy to get wrong silently. Default reduced-precision matrix arithmetic on the accelerator changed which exit a gated image took relative to a CPU reference, so reproducing a certified policy on a GPU requires disabling it. And one cloud instance's idle power drifted by several watts within a single run, which by itself was enough to fabricate an apparent saving at low frame rate that a shorter or single measurement would have missed; we now measure idle power inside every window rather than once (Appendix I).

Limits. These are accelerator measurements on data-centre hardware; they bound neither the target board nor spacecraft-grade hardware, and they exclude host CPU, memory and thermal effects. They matter here because this is the hardware class orbital data-centre proposals assume, and the contrast with a duty-cycled payload is the point — not because these

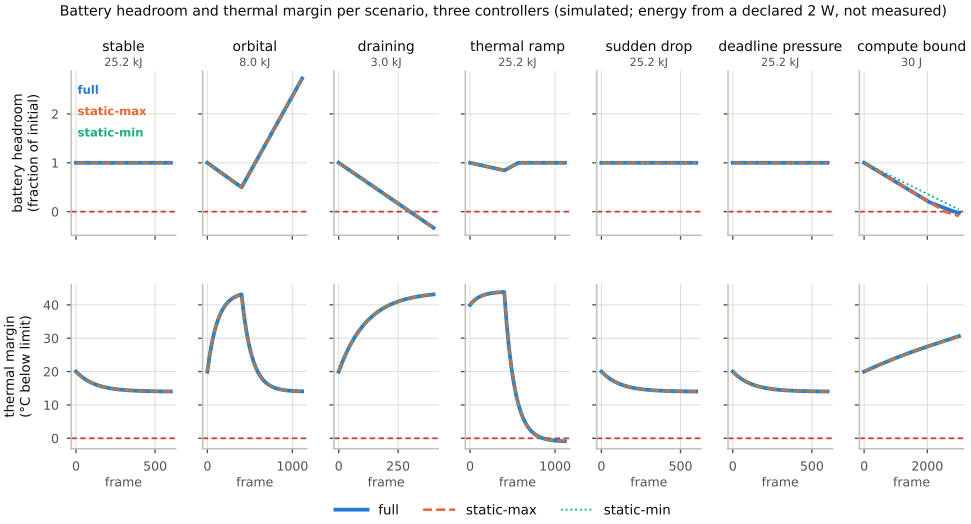


Fig. 4. Battery headroom and thermal margin per scenario, for three controllers. Both rows are plotted against their own constraint, so 0 is the battery floor above and the temperature limit below (dashed). The starting headroom is printed above each column. The three controllers coincide in every scenario except *compute_bound*; where only the dotted line is visible, all three lie on top of one another. Simulated: energy is integrated from a declared 2 W, not metered.

accelerators are themselves a candidate payload. "Saturated" here means our own small workload keeps the accelerator busy, not that a data centre's full workload does; the saving generalises to a busier accelerator only as far as that comparison holds.

B. When Adapting Pays on Energy (Section IV-C)

Setup. What adaptation can save is bounded by inference's share of the energy the platform spends anyway. We sweep that share by varying the housekeeping power on one scenario, keeping everything else about the trajectory identical, and score each controller as a share of the offline optimum for that trajectory. Below a certain share, housekeeping alone drains the battery past its floor regardless of policy; those points are excluded rather than compared.

Results. Where the energy budget binds, an adaptive controller with its price tuned for the trajectory reaches 96.7–99.0% of the optimum, against 96.7–97.6% for the best fixed configuration chosen with the same hindsight: adaptation is at best a couple of points ahead, and ties at most points on the sweep. The tightest point is also where every controller, the tuned price and the best fixed configuration included, spends through the battery floor: the ringed points of Fig. 3a are quality bought with decision-caused breaches, and the share alone would hide them. In that panel, lines that overlap coincide, and a ring marks a point whose run caused battery breaches that a run executing nothing would not have suffered; the left panel is simulated from declared power assumptions, the right panel measured hardware energy. Measurements on three accelerators say the same thing from the hardware side (Fig. 3b): adaptive routing saves 52–57% of the energy per inference while the accelerator is saturated, and at the low frame rate a satellite payload would actually use, no saving is distinguishable from zero.

Reading. Energy savings from adaptive inference are real where the accelerator itself is the energy budget, and vanish where the platform's own fixed draw is — which is the regime an Earth-observation payload at a few frames per second actually occupies. The case for adapting onboard therefore rests on meeting deadlines and memory limits under changing resources (Section IV-E), not on saving energy.

C. The Energy Price Need Not Be Tuned (Section IV-D)

Setup. Section IV-C leaves a gap: where the energy budget binds, a price tuned with foresight of the whole trajectory reaches 96.7–99.0% of the offline optimum, while an untuned default reaches only 86.7–93.5%. That tuning is worth five to ten points, and no onboard controller has that foresight. We compare three versions of the same simple decision policy, differing only in where the energy price comes from: an untuned default, the price the offline optimum selects with hindsight, and the price computed onboard by (8)–(9), with the planning horizon H set to the episode length, on the one scenario of our seven where the energy constraint actually binds.

Results. The onboard price matches the hindsight price almost exactly (97.6% of the optimum for both, executing every frame with no battery violation), where the untuned default drains the platform and loses hundreds of frames to the safety filter. The two agree because, on this trajectory, the right price is the accuracy-per-joule trade-off between two adjacent menu options, and both methods are recovering that same number by different routes. Under the rule the controller now runs (Section III-B: a declared platform draw, and the charge left divided by the frames left), the onboard price

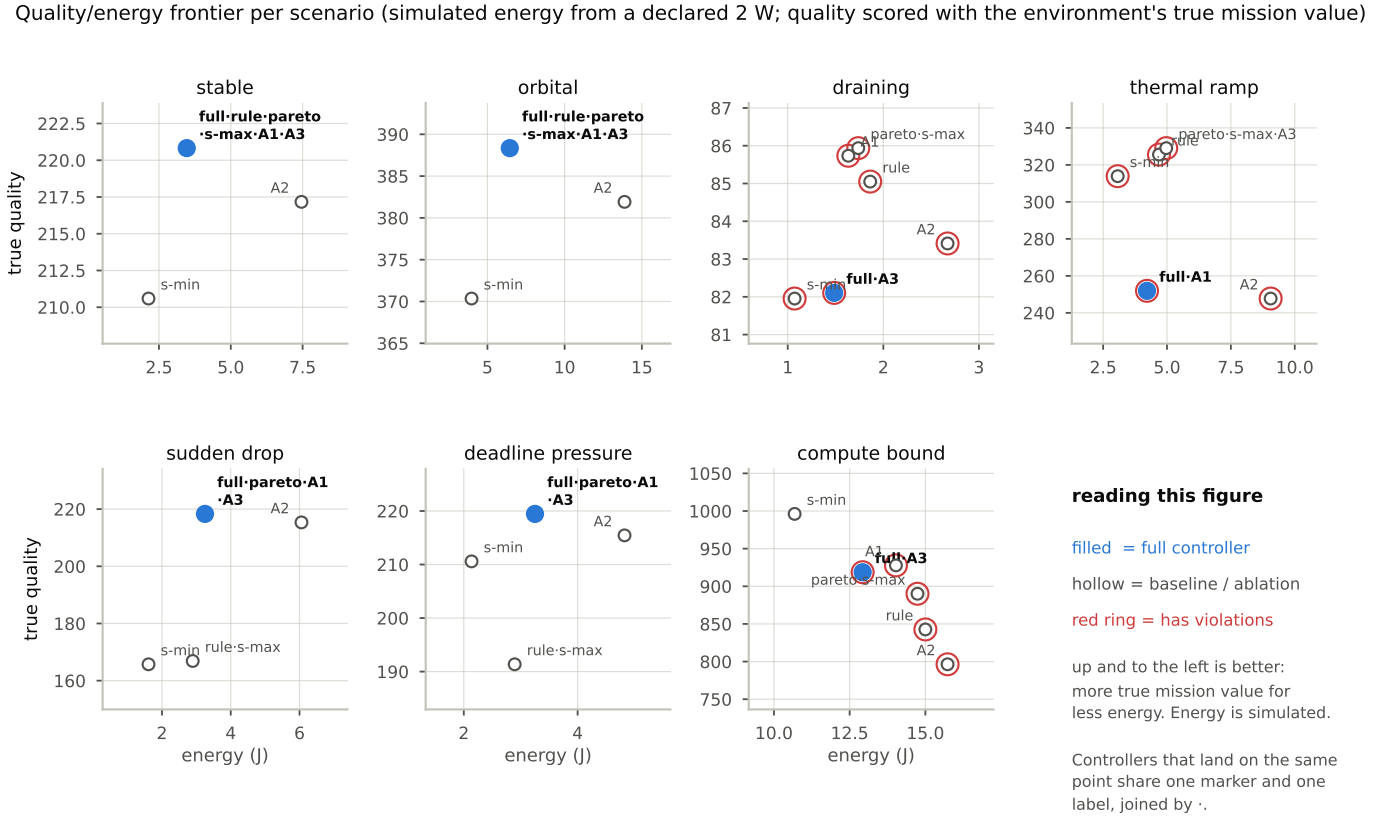


Fig. 5. Quality/energy frontier per scenario. Quality is the environment's true mission value, not the detector's estimate. Controllers that land on the same point share one marker and one label, joined by ·; a red ring marks a controller with constraint violations. Up and to the left is better. Simulated energy from a declared 2 W.

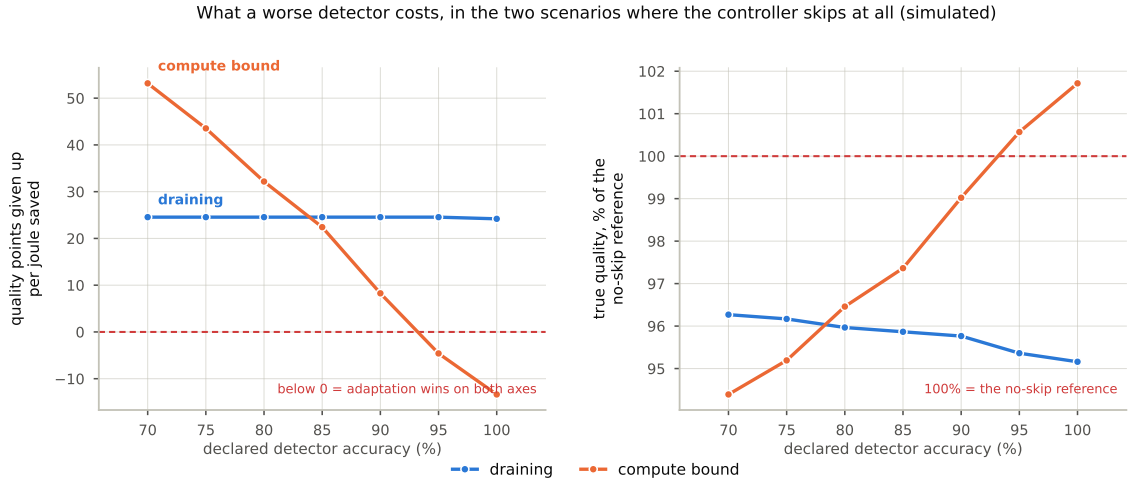
reaches 98.4%, above the hindsight price, because it can move during the episode where the hindsight price is one number for all of it.

The price and the rest of the controller do not simply add up. Adding the onboard price to the *full* controller stack (survival mode and hysteresis on top of the decision policy) reaches only 93.2% here, 4.4 points *below* the priced decision policy alone: the extra layers are buying constraint safety the price has already bought more cheaply. Under the interim rule the price was not uniformly good either: on *orbital* it lowered the full stack from 100.0 to 98.4%, and on *draining* from 65.6 to 62.6% (Table XII). Both were faults of that rule rather than costs of pricing: the measured draw took an eclipse for a permanent shortage, and a floor that idling alone loses kept the price high for nothing. The current rule corrects both; for the priced policy alone, *orbital* rises from 98.5 to 100.0% and *draining* from 65.5 to 68.6%.

What this scenario does, and does not, show. On this trajectory the onboard-priced policy spends almost every frame on a single option, essentially rediscovering the cheapest option on the menu; running the cheapest option unconditionally scores within a fraction of a point of it. So this result demonstrates that the pricing rule finds the right point on the menu without hindsight, and demonstrates little about the value of adapting, because on this trajectory the right point barely moves. A scenario whose optimum genuinely moves between menu rungs is needed to separate the two.

Phase 3 built one. On *energy_trade* (M3 menu, Table I) the offline optimum spends its allowance on three rungs, as designed, yet always running the certified rung reaches 99.4% of it. The full system reaches 98.3%, and 99.3% with a perfect value detector, so its shortfall is the detector's 10% error rather than the price. On a menu whose certified rung is within 0.9 accuracy points of full depth at 42% of its energy, adapting is worth under a point even where the optimum moves.

Limits. The rule takes two declared mission parameters, the episode length and the orbit-average platform draw, rather than measuring them onboard. Under the interim rule a shorter horizon degraded the result gracefully (from 97.6% at the full episode length to 92.8% at a quarter of it); the current rule needs the true episode length, since past it every frame is treated as the last. We treat both as mission parameters — a spacecraft knows its pass length and its power budget from mission design — rather than hidden tuning knobs, though a reader is entitled to disagree. These results are simulated trajectories over a laptop-measured menu; since the pricing rule reads its costs from that same menu, a menu that misdescribes the target board moves the price with it (Section IV-F).



Not plotted: stable, orbital, thermal ramp, sudden drop, deadline pressure — the controller never skips there, so it is identical to the no-skip reference, saves no energy, and the ratio is undefined.

Fig. 6. What a worse detector costs, in the two scenarios where the controller skips at all. Left: quality points given up per joule saved against the no-skip reference; below zero the adaptive controller wins on both axes at once. Right: true quality as a share of that reference. The five scenarios not plotted are identical to the reference and save no energy, so the ratio is undefined there. The detector's accuracy is a declared assumption, which is why this is a sensitivity and not a result.

D. Robustness to a Varying Time Budget (Section IV-E)

Setup. We vary the one resource the measured menu genuinely differs on: processing time. Energy and memory are held non-binding, on the scenario where generation matches housekeeping, so every difference comes from the time budget. During seeded bursts of pressure covering a chosen share of a 600-frame run, the deadline tightens sharply, standing for a competing task, a shorter pass, or a slower clock.

Results. Fig. 7 reports accuracy per frame. The adaptive controller matches the best possible score in every setting tested, which is close to a sanity check by construction: it confirms that the machinery around the decision rule (hysteresis, switching overhead, the controller's own decision time) never itself causes a missed frame. The most accurate fixed configuration collapses whenever the deadline falls below what it needs: at 30% of frames under pressure it drops from 90.7% to 63.5% accuracy, and to 0% once pressure is constant, because the safety filter refuses it on every pressured frame. The best fixed configuration chosen with hindsight of the whole trajectory avoids that collapse only by running the fastest option on every frame, and still concedes up to several accuracy points against the adaptive controller.

Limits. The gain over the best fixed configuration is bounded by how much the menu's options actually differ, which here is a few accuracy points at most. These deadlines and latencies are simulated and measured on a laptop CPU: Section IV-F finds the target board 6–7 times slower even at its best, so the exact deadlines swept here would need to be rescaled for the board before the same comparison applies there; re-running this sweep with board-measured costs is future work.

E. When the Menu Is Wrong About the Machine (Section IV-E)

Setup. Every cost in the menu was measured with nothing else running, and the safety filter approves a frame by trusting that number. If real execution is slower than the menu says, the filter approves work that cannot finish in time, and nothing in the controller notices, because the filter is checking the same wrong figure. This is not hypothetical on our target: the board sessions of Section IV-F measured the cheapest option running 8–9× slower than the menu claims under ambient load. We therefore sweep a range of such slowdown factors on a scenario where the deadline is the only binding constraint, and compare the menu-blind decision policy against a version that learns, per option, how wrong the menu actually is from what frames really cost.

Results. Table XI reports the two ends of the sweep and the board's own operating point. At a mild 1.5× slowdown, the menu-blind controller misses *every* contended deadline, while cost feedback cuts that to almost none for a small accuracy cost. At the factor the contended board actually presents (8–9×), the menu-blind controller still misses every deadline, and feedback holds misses to a handful too — but accuracy collapses for every variant, because at that slowdown no menu option fits the deadline at all and the only safe action left is to skip. Occasionally running the true, measured cost instead of the assumed one recovers part of that lost accuracy, at the cost of a few more missed deadlines while probing.

Reading. The accuracy column for the menu-blind controller is the trap: it looks unchanged across every slowdown because it keeps choosing the most accurate option and the menu keeps (wrongly) telling it that option fits. Counted as a hard deadline requires, its on-time accuracy is 60.4% at every slowdown: every contended frame is lost, and at a mild 1.5× cost feedback keeps 86.4% on time instead.

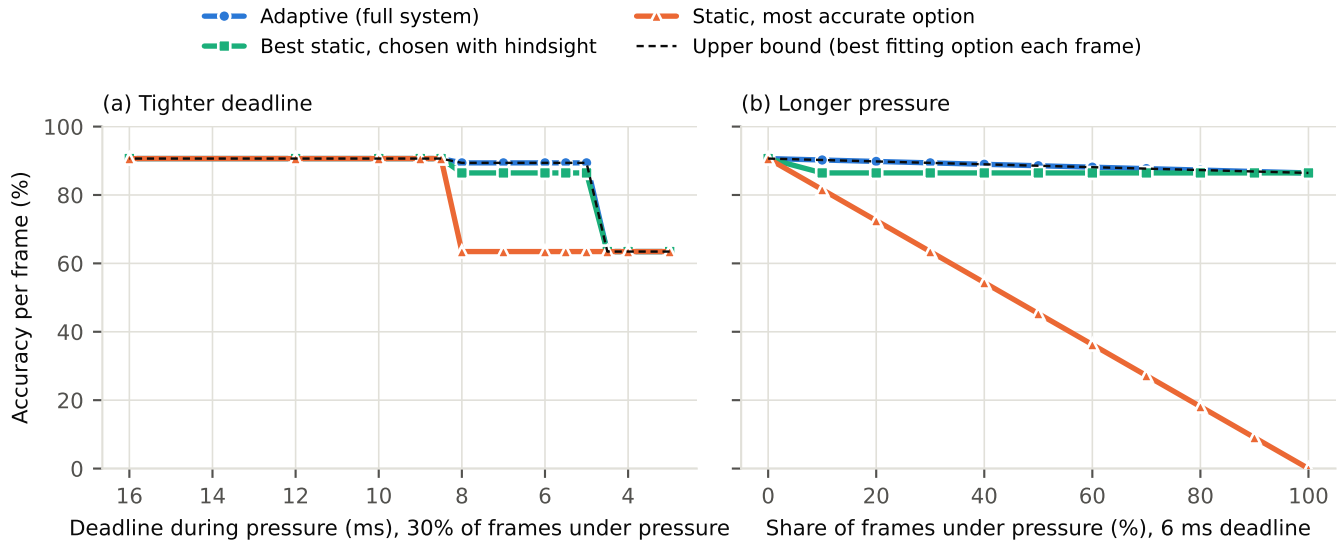


Fig. 7. Accuracy per frame (skipped and vetoed frames count as errors) as the time budget varies. (a) The deadline during pressure bursts tightens, with 30% of frames under pressure. (b) The share of frames under a tight deadline grows. The adaptive controller matches the best possible score everywhere; the most accurate fixed option collapses under pressure; the best fixed option chosen with hindsight is safe but less accurate. In (a) the deadline tightens from left to right. The dashed line gives each frame its most accurate option that fits, a per-frame bound rather than the offline optimum. Simulated trajectories, laptop-measured menu.

TABLE XI

DEADLINE MISSES (OF 200 CONTENDED FRAMES, MEAN OVER THREE SEEDS) AND ACCURACY PER FRAME AS EXECUTION OUTRUNS THE MENU. *ACC.* CREDITS EVERY EXECUTED FRAME ITS MENU ACCURACY; *ON TIME* COUNTS A FRAME THAT MISSED ITS DEADLINE AS WRONG, WHICH IS WHAT A HARD DEADLINE MEANS. THE PROBING VARIANT RUNS ONE FRAME IN TEN AT THE MENU'S OWN COST. THE TARGET BOARD'S OWN TRANSFER ERROR IS ABOUT 6.5× AT ITS QUIETEST, AND 8–9× UNDER AMBIENT LOAD, SO THESE COLUMNS BRACKET THE CONTENDED BOARD.

	1.5×			8×			9×		
Controller	miss	acc. %	on time	miss	acc. %	on time	miss	acc. %	on time
menu-blind	200	90.6	60.4	200	90.6	60.4	200	90.6	60.4
+ cost feedback	2	86.7	86.4	4	5.3	4.7	4	5.3	4.7
+ feedback, probe 1 in 10	27	88.1	84.0	25	15.4	11.7	24	14.9	11.2

What cost feedback buys is *deadline safety, not quality* — past about 3× slowdown it buys the ability to fail safely by skipping, not the ability to keep working, because no amount of correction makes an option fit a deadline that is now an order of magnitude too small. On the target board, that is the practical statement: the menu has to be re-measured on the real hardware, and drift correction is what keeps the system safe until it is.

Limits. The slowdown here is a synthetic multiplier applied to otherwise clean measurements, not a real competing process contending for cache and memory bandwidth as well as compute cycles; a controller that has decided every option is too slow also stops executing, so an estimate that caused a skip can never be corrected by observation alone, which is why the probing variant exists. Testing against a real competing workload on the board is future work.

F. Ablation: What Each Layer of the Controller Actually Buys (Section IV-H)

Setup. The full system is Survival(Hysteresis(Pareto)) — the decision policy of (7) wrapped in hysteresis and survival mode. We remove one layer at a time and re-run every scenario unchanged; the hard safety filter is never removed, so each variant measures what a component adds *on top of* a per-frame veto. A1 removes voluntary skipping (it takes the cheapest safe option instead of skipping by choice, while safety-driven skips remain). A2 removes awareness of per-frame mission value (it sees every frame as equally worth 1 and only the fixed, non-adaptive options, though it is still *scored* against the true value). A3 removes thermal awareness (it sees a temperature far below the limit, while the safety filter still sees the real one). Each variant is scored as a share of the offline optimum for its scenario.

Results. Table XII reports each variant's share of the optimum, and beside it the cheapest fixed option, which never adapts at all. On the four scenarios where no resource actually binds, the full system, A1 and A3 are indistinguishable; only A2 loses a little by being unable to spend more on the frames that are worth more. The three scenarios that do bind are more revealing, and less flattering. On the thermal-stress scenario, the thermally blind A3 scores substantially higher than the full system. On the scenario that drains fastest, the full system scores lower than A1. On the scenario where energy binds

hardest, the full system reaches 90.0% — and the cheapest fixed option, run on every frame with no adaptation at all, reaches 97.6%, beating every adaptive variant.

The two added columns isolate questions the first five cannot answer on their own. Full+price asks whether the onboard pricing rule of Section IV-D helps or hurts once it is dropped into the full layered stack rather than tested alone: it moves *compute_bound* up by three points but *draining* down by three, so it is not a uniform improvement once hysteresis and survival mode are already acting. Full+therm. is discussed below.

Reading. Every layer of survival mode buys constraint satisfaction, not quality, and it is not free. On the thermal-stress scenario most breaches are unavoidable — the die crosses its limit whether or not inference runs — so the full system's caution gives up a fifth of the achievable quality to avoid essentially one breach of its own. Where energy binds hardest, voluntary skipping, survival mode's skipping of low-value frames included, halves decision-caused battery breaches (336 without it in A1, 154 with it) for 0.9 points of quality. At the default price no variant reaches zero breaches, because a per-frame filter cannot see cumulative drain. The onboard price does: on *compute_bound* it takes decision-caused battery breaches from 454 (the decision policy alone) and 154 (the full stack) to 0, alone and inside the full stack, matching the safest fixed option without being told which one it is. Whether kept quality is worth the breaches is a mission decision, and we present no row as an outright win.

Full+therm.: a miscalibration, not a safety property. We declared in advance a variant that spends down to the cheapest still-safe option under heat instead of skipping. On the thermal scenario it recovers most of the lost quality (83.8% against 64.9%, 859 frames executed instead of 727) with the same 258 breaches, all unavoidable; the other six scenarios are unchanged. It was designed after inspecting that row and scored on the same row, so it stays a provisional variant: every other number uses the original rule, and Phase 3 judges it on a held-out thermal scenario (Section IV-I). On the M3 menu with the amended price (Table V) it again recovers most of the quality on *thermal_ramp*, 83.2% against 64.9%, but causes one decision-caused thermal breach: its extra heat brings a crossing of the limit that the ambient forces anyway forward by one frame. Under the held-out test's rule that would reject it, and we say so before the held-out run.

The system M4 runs, on development data. Table V repeats the comparison for the system as amended for Phase 3: the M3 menu, the onboard price under the current rule, cost feedback with the conformal bound, and survival thresholds as above. *Before A3* in that table is the same system with the interim price rule. Two exploratory arms, not part of M4, locate the cost of caution. Without SAVING and CRITICAL the stack reaches 100.0% on *compute_bound* with no breach, but only while the controller's own decision energy is negligible: charged the 0.55 ms cost feedback took on the board, it then breaches the floor once, unless that energy is counted in the price's platform draw, which holds 94.7% with none.

Where the ceiling comes from. The cheapest option's showing on the energy-bound scenario is the lesson of Sections IV-E and IV-G: with a narrow menu, adaptation has little accuracy to win. That argues for widening the menu, not removing the controller.

Limits. These are simulated trajectories over a host-measured menu with estimated energy: the ranking describes the control logic, not yet the board, and the survival thresholds are a declared choice that would move the energy-bound row.

Appendix G
Contention and Ablation Details

Two results in Section IV rest on sweeps whose detail would crowd the main text: how a probing controller trades misses for information under contention, and how much of the ablation's input-awareness gain depends on a detector nobody has built.

Probing costs misses to buy information. A controller that never probes cannot learn that its menu is wrong, and a controller that probes pays for the frames it spends finding out. One probe frame in ten reaches 15.4% accuracy at the cost of 25 misses, and one in twenty trades fewer misses for less information. The probe variants in Table XI exist because of that trade, and its cost is visible in the miss column. Phase 3 replaces the inflation factor with a real competing process, which is the condition under which the trade should be judged.

Input awareness pays only above a detector accuracy we have not built. A2 ignores mission value, so its 796.4 does not depend on the detector, while the full system's quality does: 943.7 with a perfect detector, 933.0 at 95%, 918.7 at the 90% default and 875.7 at 70%. Against A1, which never skips by choice, skipping on the detector's word pays outright — more quality *and* 8.2–8.5% less energy — only at 95% accuracy or better. At 90% it becomes a trade, 1.0% of quality for 7.8% of the energy, and the trade worsens as the detector does, to 5.6% of quality for 7.0% at 70%. The detector is a declared component that nobody has built or measured, which makes this the largest assumption in the paper; the sweep, regenerated in `results/detection-sweep-v2/`, is a sensitivity and not a result.

One degeneracy that is not a defect. On *compute_bound* A3 matches the full system exactly, because that scenario isolates energy and the thermal arm has nothing to do.

Appendix H
The Target Board in Full

Sections IV-F and IV-G keep the headline of each board result. This appendix gives both sections as they stood, with the evidence behind each headline.

A. Transfer Error and Platform Behaviour (Section IV-F)

The results of Sections IV-B–IV-E were obtained on a development host and in simulation. The board sessions had no power meter or active cooling, so together they measure latency, memory, thermal behaviour and controller decision time on ARM, and they measure no energy at all.

The menu is optimistic by about 6.5× on the board, measured floor against floor. The cheapest option in the

TABLE XII

SHARE OF THE OFFLINE OPTIMUM (%) WITH ONE CONTROLLER LAYER REMOVED, AND FOR THE CHEAPEST FIXED OPTION RUN ON EVERY FRAME. **FULL, A1, A2 AND A3 ALL USE THE DEFAULT ENERGY PRICE** $\lambda = 1$. FULL+PRICE IS THE SAME FULL STACK WITH λ COMPUTED ONBOARD BY (9) UNDER THE INTERIM RULE (SECTION IV-D); FULL+THERM. IS THE PRE-DECLARED THERMAL-RECOVERY VARIANT DISCUSSED BELOW. THE HARD SAFETY FILTER IS PRESENT IN EVERY COLUMN. HIGHER IS BETTER, BUT READ IT WITH THE DECISION-CAUSED BREACHES, THOSE BEYOND A RUN THAT EXECUTES NOTHING: ON *COMPUTE BOUND*, 154 BATTERY BREACHES FOR FULL, 336 FOR A1, 619 FOR A2, 154 FOR A3, 0 FOR FULL+PRICE, 154 FOR FULL+THERM. AND 0 FOR CHEAPEST; ON *THERMAL RAMP*, ONE THERMAL BREACH FOR A3; NONE ANYWHERE ELSE.

Scenario	Full	A1	A2	A3	Full+price	Full+therm.	Cheapest
stable	100.0	100.0	98.3	100.0	100.0	100.0	95.4
sudden drop	100.0	100.0	98.6	100.0	100.0	100.0	75.9
orbital	100.0	100.0	98.3	100.0	98.4	100.0	95.4
deadline pressure	100.0	100.0	98.2	100.0	100.0	100.0	96.0
draining	65.6	68.5	66.6	65.6	62.6	65.6	65.5
thermal ramp	64.9	64.9	63.8	**84.7**	64.9	**83.8**	80.8
compute bound	90.0	90.9	78.0	90.0	93.2	90.0	**97.6**

TABLE XIII

THE SAME OPTION, THE SAME CONTROLLER AND THE SAME RATE, MEASURED TWENTY HOURS APART WITH THE DESKTOP RESIDENT IN BOTH CASES. ONLY THE MACHINE'S AMBIENT LOAD DIFFERED (3.2 AGAINST 4.2). THE FLOOR REPRODUCES; THE MIXTURE STATISTICS DO NOT. THE HEADLINE RATIO IS THE FIRST OF THE TWO BOTTOM ROWS, QUARTILE AGAINST QUARTILE ON THE RE-MEASURED HOST, THE SECOND IS THE FIGURE AN EARLIER DRAFT OF THIS PAPER PRINTED AND IS KEPT ONLY SO THE CHANGE IS VISIBLE.

	Session 1	Session 3	Change
Minimum (ms)	9.37	9.24	−1.4%
p_{10} (ms)	9.45	9.34	−1.2%
p_{25} (ms)	9.50	9.41	−0.9%
Median (ms)	9.57	14.86	+55.3%
Mean (ms)	12.37	14.33	+15.9%
ℓ_{95} (ms)	22.25	24.92	+12.0%
p_{25} / host p_{25}	**6.56**×	**6.50**×	−0.9%
superseded: p_{25} / menu med.	6.43×	6.37×	−0.9%

TABLE XIV

CLOSED LOOP ON THE TARGET BOARD, THREE TRIALS OF 120 FRAMES AT EACH RATE, SYNTHETIC IMAGES, ONE FIXED PATH PER FRAME, NO FAN, SESSION 1, AMBIENT LOAD AVERAGE ABOUT 3.2. PERCENTILES ARE POOLED OVER ALL 360 EXECUTED FRAMES OF A RATE; PER-TRIAL ℓ_{95} RANGES ARE 19.8–23.8 MS AT 1 FPS AND 23.5–27.9 MS AT 0.2 FPS, AND THEY OVERLAP. THESE ARE DESCRIPTIONS OF A CONTENDED BOARD AT A STATED LOAD, NOT MENU-TRANSFER FIGURES: SEE TABLE XIII. ENERGY IS ABSENT BY DESIGN: NO METER WAS CONNECTED.

	1 fps	0.2 fps
Mean latency (ms)	12.37	12.53
Median (ms)	9.57	9.62
ℓ_{95}, pooled (ms)	22.25	25.00
Maximum (ms)	41.15	38.04
Resident memory (MB)	315.3	315.3
Controller decision, mean (ms)	0.023	0.022
Deadline misses	0	0
Violations (all four kinds)	0	0
Temperature, median / max (°C)	55.5/59.4	47.7/62.8
Throttled samples	0/474	0/1891

frozen menu, exit 1 at 64 pixels, runs on the Raspberry Pi at a first quartile of 9.50 ms in one session and 9.41 ms in another twenty hours later, across a reboot, a resynchronisation and a 9 °C difference in board temperature — a floor reproducible to 0.9%. Against the same option's first quartile on the re-measured development host (1.45 ms), that is 6.56× and 6.50×: the headline we quote, and the one the rest of this section relies on.

The comparison is made quartile against quartile deliberately. Across statistics the same two sessions give 6.56× and 6.50× on p_{25}, about 7.8× on the minimum, and 6.0× against 9.4× on the median: the median moves 56% between sessions on ambient load alone, while the floor moves 1%. A menu entry claims what the work costs, so the statistic that tests it is drawn from uncontended frames, and it must be the same on both machines.

One caveat travels with the ratio: the 0.9% belongs to the board's floor, not to the ratio, whose host denominator reproduces only to about ±7% on re-profiling.

Table XIII is the evidence behind that rule. Its last row keeps the ratio an earlier draft printed (6.43×, against the frozen menu's median) so that the correction is auditable rather than asserted.

Even taken at the floor, the gap is far larger than our own robustness analysis says a menu-blind controller survives. Our contention study reports that at a real-to-predicted latency ratio of 1.5 a controller that trusts its menu misses 200 of 200 contended deadlines, and that at the 8–9× end of the sweep no option fits, so every variant without probing falls to 5.3% accuracy (Table XI). The board is at about 6.5 when it is quiet and worse whenever it is not. Cost feedback — a controller that corrects its menu against what it observes — is therefore not a robustness extra on this hardware but a precondition for the safety margins of (6) to mean anything. No deadline was missed in the first session only because the *stable* scenario allows 50 ms and 12.5 ms fits inside it; against the 9 ms budget the contention experiment uses, this menu would not fit at all.

Per-frame cost does not depend on pacing. Table XIV repeats the closed loop at 1 and 0.2 frames per second. Dropping the duty cycle fivefold, across runs 25 minutes apart, moved the mean latency by 1.3% and the resident memory not at all. The per-trial ℓ_{95} ranges at the two rates overlap, so the difference between them is trial-to-trial scatter and not a property of the frame rate. The transfer error is thus a property of the hardware rather than an artefact of how hard the board was pushed. It also confirms at the board what Section IV-B measured at the accelerator: the cost of one inference is fixed, so lowering the frame rate does not make a frame cheaper — it spreads the platform's fixed draw over fewer of them.

What a decision costs on ARM depends on the layers.

In the standalone benchmark of the first three sessions a decision takes 0.03 ms with a 99.9th percentile of 0.116 ms, about 15× inside the $\delta_\ell = 2$ ms margin of (6) and 0.13% of a ∼22 ms frame; one variant's isolated maxima exceed the margin reproducibly (Appendix D). That benchmark stops before cost feedback. Inside the paced loop of session 4, run means are 0.02–0.03 ms for a fixed option, 0.08–0.13 ms for the adaptive policy and 0.35–0.55 ms with cost feedback, which reached 1.19 ms for a single decision; with a 1.25 ms switch that crosses the margin, on 30 of 360 frames at the governor's low clock and 2 of 360 at full clock. The feedback layer now reuses its corrected options instead of rebuilding them every frame; its board cost is re-measured in the next session.

Thermal behaviour without cooling. Across the first three sessions there were zero throttled samples against a soft limit of 80 °C, with a steady state near 63 °C and a hottest moment of 68.1 °C on the most heavily loaded board; session 4, headless and also without a fan, peaked at 65.2 °C with no throttled sample. At 0.2 fps the governor drops the ARM clock to 700 MHz between frames and returns to 1.8 GHz per inference, which is why the slower run is the cooler one and why its tail latency is marginally worse. Saturated runs and 10 fps were deliberately not attempted without a fan.

How large could inference's share of a frame be? An order-of-magnitude bound. We cannot measure it, but we can bound it well enough to know whether it matters. Taking a published measured Raspberry Pi 4 base draw of 1518 mW [32], our own declared 2.0 W active increment, the simulator's 5 s frame period, and — pessimistically — the largest per-run mean latency any board session produced (26.95 ms, the cost-feedback-off arm), one inference accounts for 0.7% of a frame's energy. The point is not that figure but its robustness: halving the base power and raising the active increment to 5 W, wider than any plausible mismatch between that published figure and ours, moves it only to 3.4%; doubling the base and dropping the increment to 1 W gives 0.2%. Across that whole range inference stays a few percent of a paced frame, which is the conclusion Section IV-B draws from the accelerators, reached independently on the board's own latencies. This is *estimated* from a published figure and declared constants, not measured on our board, and it is not a substitute for the meter: it bounds the share, and says nothing about what the platform draw itself could be reduced to.

What these sessions do not show. No session shows energy: the run summaries' 3.13 J is the declared 2.0 W times the measured latency, and enters no claim. The first three do not show accuracy: their images were synthetic, so the 2.71 points in Table VI are ascribed by the menu, not observed. And with the desktop resident (load 3.2–4.3), their absolute latencies are upper bounds; the 6.5× floor ratio is the exception by construction. They predicted that a headless repeat would hold the floor near 9.4 ms while the mixture collapsed toward it.

Session 4: headless, real images, and the clock. The prediction held for the floor and failed for the mixture. Headless and on real EuroSAT images, the floor ratio is 6.60× (board p_{25} over host p_{25}), against 6.56 and 6.50× with the desktop. The slow tail did not collapse: at full clock, going headless moved floor, median and ℓ_{95} by under 3% against the first session, so the desktop was not what produced it. The CPU governor was. For the first twenty minutes the clock sat mostly at 600–900 MHz and every arm ran about 1.9× slower; afterwards it sat at 1.8 GHz. Deadline misses followed the clock, not the controller: 15 and 12 for the two adaptive arms in the slow block, none for any arm at full clock, and none for the cheapest option in either. The session also measured accuracy on the target: on all 2,560 frames, across 7 options, the board predicted the host's class and took the host's exit, so the host's accuracy carries over to the board. The paced images covered only 2 of the 10 classes, which is why the all-class accuracies remain the host's.

B. Cost Feedback on the Target Board (Section IV-G)

Cost feedback, measured against a real transfer error. Section IV-E drove the controller with a simulated slowdown; a second board session drives it with the board's own, real one. Table VI compares the adaptive controller with cost feedback disabled and enabled, interleaved so a board warming up over the session cannot be mistaken for a difference between arms. Treating each trial as the unit of comparison (frames within a trial are not independent), every trial with feedback beats every trial without it on mean latency (exact one-sided permutation test over six trials per arm, $p = 0.0011$, the smallest that test can give). The table's first session ran at desktop load about 3.2; accuracy and quality there are the values the menu attributes to the options chosen, not observed measurements.

The mechanism is visible in the controller's own choices. Without feedback it stayed on one adaptive option all session: the filter budgeted it at about 6 ms, but the board executed it at 26–27 ms, a 4–4.5× prediction error (a mean against a budgeted percentile on a gated option, so not comparable to the 6.5× floor ratio). With feedback on, it spread across seven of the eight options and roughly halved that error by learning which options this board can afford.

One caveat: the frames were synthetic, and a confidence gate routes on image content that noise lacks, so the gated option took its deep exit far more often than on real images. That inflates both the prediction error and the improvement; the fixed-option 6.5× ratio is unaffected. Session 4 confirmed both halves on real images. The gated option left at its first exit on 320 of 400 real frames, exactly as on the host, against 60 of 720 synthetic ones. And cost feedback still helped, but only while the board ran slower than the menu: at the low clock it cut mean latency by 13.8% and misses from 15 to 12, for 2.22 points of measured accuracy ($p = 0.008$, McNemar), and at full clock it changed nothing.

From an average to a bound. The filter tests a 95th percentile, so an average settles near the board's mean and understates the tail. A point estimate of the 95th percentile was exceeded on 6.9% and 7.0% of frames for the two options the controller ran.

Against the baseline that does not adapt at all. Cost feedback makes the adaptive controller better than it was; it does not make it better than the simplest available policy. In the first board session, on the same scenario, rate and menu but not interleaved with these runs, always running the cheapest option missed *no* deadline at all (Table VI); a later session that ran it interleaved with both feedback arms again recorded none. What the corrected adaptive controller buys over that simple baseline is about 1.5 points of accuracy, paid for in 23 missed deadlines and several milliseconds of extra mean latency. That is a real trade, and on this board it is not obviously worth making. The cause is the menu, not the controller: with only a narrow accuracy spread on offer (Section IV-E), adaptation has little to win and a real latency tail to risk winning it. We report this rather than a more flattering comparison because it is the question a mission designer would ask first, and because it is the strongest argument in this paper for a wider, certified menu.

A boundary worth stating plainly. These board sessions all use the scenario where nothing binds, which is exactly where a cheap fixed option should win. Our simulated scenarios show the comparison inverts once a resource genuinely binds: the full adaptive system reaches the optimum on scenarios with a sudden resource drop or deadline pressure, where the cheapest fixed option gives up noticeably more (Section IV-H). The honest claim is therefore not that adaptation fails to pay on this board, but that it has not yet been tested where it should.

Appendix I
Measurement Pitfalls

Three of our findings are about measurement rather than about the system, and each one first appeared as a plausible result that turned out to be an artefact. They are collected here because they generalise past this paper: any group profiling a gated network on an accelerator, or profiling a menu on one machine to execute it on another, can meet all three.

Do not extrapolate the paced case from the saturated one. Fitting (12) with E_{dyn} taken from saturated windows predicts the paced total within 1%, which looks like a validated model and is not: the total is dominated by P_{idle}/f, and the same fit underestimates the *measured* paced above-idle energy by 57–88% on every platform. An isolated frame costs more above idle than a saturated one, consistent with per-frame power-state transitions. We therefore withdrew an earlier always-on saving estimate derived this way, and we measure each offered rate instead.

Numerics change gated routing. With PyTorch's default TF32 convolutions on Ada/Ampere GPUs, 0.18% of labels and 0.31% of exits differ from the CPU reference, because gated routing turns a small numerical difference into a different execution path. With TF32 disabled the GPU reproduces the CPU routing exactly (maximum confidence difference 3×10^{-5}), so reproducing a certified policy is a deployment requirement, not a preference.

Idle drift fabricates a paced saving. On a cloud L4 the idle board power wandered between 29.2 and $37.4\,\text{W}$ inside a single 39-minute run, independently of which option was executing. At 10 frames per second, $1\,\text{W}$ of such drift moves each frame by $100\,\text{mJ}$, more than the entire dynamic cost of full depth, and it produced an apparent 7–13% paced "saving". Our analysis now attributes each contrast to idle shift explicitly and flags it when that share exceeds one half; it is 96–110% for those paced contrasts and 5–8% for the saturated ones, which is why we report the latter and not the former.

Why a menu-transfer error must be read off the floor. A menu entry is a claim about what the work costs. A board's mean, median and ℓ_{95} are a *mixture* of uncontended frames drawn from the hardware's floor and a separate population that waited, so their value tracks how busy the machine was rather than what the work costs. Between two sessions twenty hours apart, with the same option, controller and rate, our board's median moved by $+55.3\%$ and its mean by $+15.9\%$ while its minimum moved by -1.4% — on nothing but the machine's ambient load rising from about 3.2 to 4.2. Across statistics those two sessions give $6.56\times$ and $6.50\times$ on p_{25}, about $7.8\times$ on the minimum, and $6.0\times$ against $9.4\times$ on the median. The rule we follow is therefore to compare the statistic drawn from uncontended frames, and to use the same statistic on both machines; the mixture statistics are reported as descriptions of a contended board, quoted with the load average beside them, and never as transfer figures.

Appendix J
Phase 3 Plan, Milestones and Threats to Validity

Section IV-I states the held-out evaluation's rules in one paragraph. This appendix gives that section as it stood: what is established, the measurement plan behind the remaining milestones (Table XV), the pre-registration and the threats to validity.

What is established, and what is not. Four sessions on the payload have settled more than a plan would have: latency and its floor, resident memory, thermal behaviour under no cooling, controller decision time on ARM, the cost-feedback comparison of Section IV-G, and host–board agreement on every frame of real images are measurements on the target. What remains is bounded and specific. Every *controller* result outside those sections is a closed-loop simulation driven by costs measured on a development host. Every energy figure outside Section IV-B is a declared active power over a measured latency, labelled as an estimate wherever it appears; the accelerator energy is real but comes from data-centre boards, which bound neither the target nor spacecraft hardware. And accuracy on the target, though now measured, covers two of the ten classes. The order below puts the results that need no instrument first.

Target and instrument. The onboard target is a Raspberry Pi 4B ($8\,\text{GB}$); its VideoCore GPU cannot run the network, so inference is ARM CPU work and the GPUs remain the saturated-regime reference only. Board energy is measured at the input with an inline USB-C meter that logs to a host

with timestamps, samples at 10 Hz or better and resolves 1 mA, since a display-only meter cannot be joined to inference windows. The protocol of Section IV-B transfers unchanged except for the energy counter, which the meter replaces.

Measurement protocol. Runs are paced at each offered rate (0.2, 1 and 10 frames per second) rather than extrapolated from saturated windows, because the two regimes differ by exactly the effect under study. Idle and active windows alternate, at least three trials per cell with an idle window on each side, so that board idle drift can be separated from the difference being measured; we report the idle-shift share for every contrast and discard any contrast it dominates. Torch is pinned to the two threads the menu was profiled with, temperature and throttling are sampled at 1 Hz for the whole session, and any run containing a throttled sample is void rather than reported. Energy per frame is (active energy $-$ idle power $\times$ active duration) divided by frames, the same subtraction used on the accelerators.

What the remaining measurements decide. Controller decision time is already measured on ARM (Section IV-F). Cost feedback is half-closed: it was driven by the board's real transfer error, but under ambient rather than declared contention. The claim that adaptation buys deadlines rather than joules still rests on a declared housekeeping power; a measured Pi idle draw is what moves the boundary in Fig. 3.

Accuracy on the target, which needs no instrument. Session 4 scored real images on the board, frame by frame against the development host through the same gates, and found exact agreement on every frame, with the cheapest fixed option interleaved with both cost-feedback arms and the desktop stopped (Section IV-F). What remains is coverage: the paced images were the first of a class-sorted split and covered two of the ten classes, so the next session draws a stratified sample of all ten, pins the CPU governor, and times the conformal feedback layer.

Power state as an action. Our measurements point at the payload's fixed draw as the energy that matters on a paced frame, and Section III-A states that the current controller cannot touch it. Phase 3 closes that gap in the order the evidence allows. The meter first gives the board's idle, active and boot energy. A Raspberry Pi has no low-power suspend state, so powering down means switching the payload off at the spacecraft's power switch and paying a boot on the next wake-up. The decision is therefore made per observation window rather than per frame: stay awake through a gap, or switch off and pay the boot. We add that decision to the controller with the measured costs, under the same veto-only filter, and report it against the always-on policy on the held-out scenarios below.

Table XV sets out the five milestones, their dates, the exit criterion of each, and where each stands.

Held-out evaluation. Development has used the same scenarios and seeds throughout, so the controller settings have seen them. Phase 3 therefore closes with a single evaluation on held-out seeds and at least one scenario not used in development, run once, with the thresholds and prices frozen beforehand and the result reported whichever way it falls.

That evaluation carries one pre-registered test. The thermal-recovery variant of Section IV-H was designed on *thermal_ramp* and scored there, so it is judged on a thermal scenario built for the held-out set: *it is adopted only if it gains quality there while causing no more decision-caused thermal breaches than the original arm.*

The rules are written down before any held-out run, in two stages: how every setting is derived (committed on 2 October), then the values those rules produce (by 8 October). They fix ten seeds derived from a published phrase, two held-out scenarios, and the predictions: no decision-caused breach; the onboard price within two points of the hindsight price; quality at least that of the cheapest fixed option; the thermal test above; and a test of power state as an action if the meter arrives. Three amendments, each made before any held-out run with its reason and its effect on development results recorded, replaced the interim price rule and cost-feedback rule with those of Section III-B and fixed the five-rung menu of Table I, on host costs unless the board re-profile lands before the values are frozen. Development data already says where the predictions are weakest (Table V): quality falls below the cheapest fixed option on *thermal_ramp* on every development seed and once on *compute_bound*, and the thermal variant would be rejected. We expect both on the held-out set, and report them whichever way they fall.

Why the price and cost-feedback rules changed. The interim price rule took an eclipse for a permanent shortage and kept the price high for a floor that idling alone loses (Appendix F); the interim cost-feedback rule learned an average where the filter tests a percentile (Section IV-G).

Threats to validity we already know of. The offline optimum is loose where the safety filter forbids frames whatever runs; the ceiling matched to the filter (Section III-C) removes that looseness, and both are reported. Where the battery floor is lost whatever runs, any controller that executes brings the loss forward, so the decision-caused count charges it breaches: in the sweep behind Fig. 3, every arm, fixed options included, does so once housekeeping alone breaches the floor, by fewer frames the smaller inference's share of the draw. The price's unreachable-floor rule then spends like the most expensive option. The first prediction is weakest in that regime. The survival thresholds are a declared choice: time-to-floor at the housekeeping draw, capped at a share of the starting reserve. Before the cap, the smallest-battery scenario sat below SAVING for its whole run, and results taken there described a throttled system; they have been re-measured under the cap (Section IV-H), but a different cap would move them again. The mission-value detector is a declared placeholder with a 10% error rate; the sweep that bounds its influence (Appendix G) shows the conclusion it supports is conditional on that placeholder, and no claim rests on it until a real detector exists. The Raspberry Pi is not radiation-tolerant: it flies on BUPT-1 as a research payload in low Earth orbit, and no result here says anything about single-event upsets, total dose or a multi-year mission. Our claims are about the COTS

TABLE XV
PHASE 3 MILESTONES (1–14 OCTOBER 2026), AS PLANNED AND AS THEY STAND ON 4 OCTOBER. EACH MILESTONE HAS AN EXIT CRITERION THAT IS A NUMBER REPORTED WHICHEVER WAY IT FALLS, NOT A CLAIM TO BE CONFIRMED.

	Dates	Work	Exit criterion	Status on 4 October
M1	1–3 Oct	Board session on real held-out images; cheapest fixed option interleaved with both cost-feedback arms	measured board accuracy; host–board agreement per frame	done early, in session 4 (28 Sep): agreement on every frame; accuracy on 2 of 10 classes, all ten pending
M2	1–5 Oct	Inline meter; paced 0.2, 1, 10 fps with idle windows	measured idle, per-frame and boot energy on the Pi	blocked: the meter has not arrived; energy stays estimated
M3	4–8 Oct	Certified five-rung menu re-profiled on the board; power state added as an action	Fig. 3 redrawn from measured idle	menu built on host costs (Table I); board re-profile pending; power state waits on the meter
M4	9–12 Oct	Held-out seeds and scenario, thresholds and prices frozen, run once	all five indicators, adaptive vs. every fixed option	rules pre-registered (2 Oct) and amended before any held-out run (3 Oct); values frozen by 8 Oct
M5	13–14 Oct	Final report and live board demonstration	reproducible build; claims checker passes	claims checker passes on the current sources

ARM class in the low-duty-cycle regime, and a mission-grade port would target a radiation-tolerant part — for which the methodology, not the numbers, is what transfers: measure the target and correct the cost model on it. Finally, all platform parameters are declared assumptions for a 3U-class spacecraft, not measurements of a flight system, and they are replaced wholesale once a board and a meter exist rather than tuned toward a desired result.

APPENDIX K
CORRECTIONS LOG

This work has been revised several times, and some revisions changed what we claim. Rather than leave each correction attached to the result it touched, where a reader meets it before knowing what the current claim is, they are collected here in date order. Each entry states what was claimed, what replaced it, why, and where the evidence is. Every figure named below is in the current paper; nothing was withdrawn without a replacement.

- **The retraining corrections (2026-09).** The frozen menu comes from a single training run whose final head at 48 pixels was weak (69.3%), which flattered gated inference. Retraining under a protocol frozen in advance corrected three claims we had made: that adaptive inference is generally more accurate than full depth (it is not; it costs 1.70 ± 0.38 pp at 64 pixels, negative for every seed); that the 48-pixel head was representative (training at both resolutions repairs it, $79.4 \pm 7.4\% \rightarrow 92.5 \pm 0.7\%$); and that random-split accuracy describes onboard performance (a geographic split costs about five points). The results are in Section III-D.
- **The always-on saving estimate, withdrawn (2026-09).** An early estimate of the paced saving was derived by fitting the frame-energy model on saturated windows. That fit predicts the paced *total* within 1% while underestimating its above-idle energy by 57–88%, so the estimate was withdrawn and each rate is now measured directly (Appendix I).
- **The uncapped survival thresholds (2026-09-20).** An earlier version of the ablation, run under uncapped thresholds, reported A2 coinciding with the cheapest static option and the full system incurring no breach on *compute_bound*. Both were artefacts of a controller held in SAVING for the whole episode, not properties of the design. The capped thresholds of Section IV-H replace them.
- **The last-trial-only accuracy figures (2026-09-21).** The cost-feedback table first quoted accuracy and mission-weighted quality from the final trial alone, giving a 3.00 pp accuracy cost. Summed across all trials the figure is 2.71 pp. The deadline-miss counts had the same defect and were corrected first; the accuracy and quality columns followed.
- **The median as the transfer-error headline (2026-09-21).** An earlier draft headlined the board's median, on the argument that it would be stable when the board was rerun quietly. The third session measured the median moving 55% between sessions on ambient load alone, while the floor moved 1%, so the argument was wrong and the floor replaced it (Appendix I).
- **The transfer error as $6.43\times$ and $6.37\times$ (2026-09-22).** Those figures divided a board *floor* by a *median* on a host whose processor the menu never recorded — a mean-against-percentile comparison of exactly the kind this paper argues against. Replaced by p_{25} against the re-measured host's p_{25}, about $6.5\times$. The conclusion did not change; the method did. Table XIII keeps the old row, labelled historical, so the change is visible.
- **The gated option's transfer error as $9.9\times$ (2026-09-23).** That number applied the superseded rule above to `adaptive_r64`. Under the current rule it is $14.2\times$, and Section IV-G explains why neither figure should be read beside the fixed option's $6.5\times$.
- **The grid dynamic program (abandoned).** Tried first for the offline optimum and abandoned: one frame moves the battery by a few millijoules against grid cells of a fraction of a joule, so discretisation swamped the quantity being bounded (Appendix C).
- **Dual ascent on the energy price (abandoned).** Implemented first, and worse than both the closed form and the untuned default (Appendix C).